



CÓDIGO DRAGÓN

Taller 100 % guiado de Fundamentos de Java

Scanner · Decisiones · Ciclos · Arrays · Clases y Objetos

Primer semestre de universidad

7 niveles + jefe final · 865 XP · 5 horas

Con protocolo de uso responsable de Inteligencia Artificial

"Datos que entran, decisiones que se toman, cosas que se repiten, listas que se recorren y objetos que existen."

CONTENIDO

Si el índice aparece vacío, presione Ctrl + A y luego F9 para actualizarlo.

 CONTENIDO	2
Taller 100% guiado de Fundamentos de Java	7
 LA HISTORIA (léela, es parte del taller)	7
 ¿QUÉ VAS A SABER HACER AL TERMINAR?	7
 REGLAS DEL JUEGO	8
Sistema de puntos (XP)	8
Insignias	8
 PROTOCOLO COPILOTO: usar IA sin volverte inútil	9
✅ Permitido	9
❌ Prohibido	9
🔒 La Regla de los 60 Segundos	9
🎯 El filtro obligatorio de este taller	9
📅 Bitácora de IA (obligatoria, Anexo G)	10
 KIT DE HERRAMIENTAS	10
NIVEL 0 — CAMPO DE ENTRENAMIENTO ✂	10
<i>"Antes de crear un dragón, aprende a decir hola"</i>	10
 CARGA RÁPIDA	10
La anatomía de un programa Java	10
println vs print	11
Variables: la caja con nombre y con sabor	11
Operadores	12
Concatenar texto con +	12
Comentarios	12
 TE LO MUESTRO — La ficha de estadísticas	13
 LO HACEMOS JUNTOS — La bolsa de oro	14
✅ CHECKPOINT NIVEL 0	14
NIVEL 1 — LA CREACIÓN DEL HÉROE 🧑	15
Entrada de datos con Scanner	15
 CARGA RÁPIDA	15
Los 3 pasos del Scanner (siempre los mismos)	15
Los métodos de lectura	15
💀 LA TRAMPA DEL SALTO DE LÍNEA (el bug #1 de todo estudiante de Java)	16
 TE LO MUESTRO — La creación del héroe	16
 LO HACEMOS JUNTOS — La tienda del herrero (10 XP)	18

 AHORA TÚ — El registro del gremio (20 + 15 + 15 XP).....	18
 CAZA DE BUGS #1 (25 XP)	19
 ESTACIÓN IA — Nivel 1	19
 CHECKPOINT NIVEL 1 — XP: _____ / 95.....	20
NIVEL 2 — EL CAMINO SE BIFURCA ✂	20
Decisiones: if, else if, else y switch	20
 CARGA RÁPIDA.....	20
if simple y if / else	20
La escalera: if / else if / else	21
Operadores de comparación y lógicos.....	21
 LA TRAMPA MÁS IMPORTANTE DE ESTE NIVEL: comparar Strings.....	22
switch: para valores exactos	22
 TE LO MUESTRO — El selector de clase y el golpe crítico	23
 LO HACEMOS JUNTOS — El portero de la mazmorra (10 XP)	24
 AHORA TÚ — La tienda de pociones (20 + 15 + 15 XP).....	25
 CAZA DE BUGS #2 (25 XP)	26
 ESTACIÓN IA — Nivel 2 · DUELO DE CASOS BORDE	27
 CHECKPOINT NIVEL 2 — XP: _____ / 95.....	27
NIVEL 3 — COMBATE POR TURNOS ✂🐉	28
Ciclos: while, do-while y for	28
 CARGA RÁPIDA.....	28
while — <i>"mientras se cumpla, repite"</i>	28
do-while — <i>"hazlo al menos una vez, luego revisa"</i>	28
for — <i>"repite un número conocido de veces"</i>	29
Contadores, acumuladores y atajos.....	29
break y continue (úsalos poco)	29
 TE LO MUESTRO — El combate contra el dragón	30
 LO HACEMOS JUNTOS — El entrenamiento (10 XP)	31
 AHORA TÚ — El torneo del gremio (20 + 15 + 15 XP)	32
 CAZA DE BUGS #3 (25 XP)	32
 ESTACIÓN IA — Nivel 3 · CAZA DEL CICLO INFINITO	33
 CHECKPOINT NIVEL 3 — XP: _____ / 95.....	33
NIVEL 4 — EL INVENTARIO 📦	34
Arrays: declarar, llenar y recorrer	34
 CARGA RÁPIDA.....	34
Las dos formas de crear un array.....	34
 Los índices empiezan en CERO.....	34

.length — cuántas casillas tiene.....	35
Recorrer un array	35
Modificar una posición	35
👁️ TE LO MUESTRO — El inventario del héroe	35
👉 LO HACEMOS JUNTOS — Los daños del combate (10 XP)	37
💪 AHORA TÚ — La bitácora de la mazmorra (20 + 15 + 15 XP)	37
🐛 CAZA DE BUGS #4 (25 XP)	38
🚗 ESTACIÓN IA — Nivel 4 · EL PREDICTOR.....	38
✅ CHECKPOINT NIVEL 4 — XP: _____ / 95.....	38
NIVEL 5 — EL SALÓN DE LA FAMA 🏆	39
Arrays + ciclos: máximos, búsquedas y estadísticas	39
🧠 CARGA RÁPIDA.....	39
Patrón 1 — Suma y promedio.....	39
Patrón 2 — Máximo (y su posición)	39
Patrón 3 — Búsqueda	40
Patrón 4 — Conteo condicional	40
Arrays paralelos (y por qué son un problema).....	40
👁️ TE LO MUESTRO — El salón de la fama del gremio	41
👉 LO HACEMOS JUNTOS — El daño máximo (10 XP)	42
💪 AHORA TÚ — El análisis de la temporada (20 + 15 + 15 XP)	43
🐛 CAZA DE BUGS #5 (25 XP)	43
🚗 ESTACIÓN IA — Nivel 5 · TÚ ERES EL PROFESOR.....	44
✅ CHECKPOINT NIVEL 5 — XP: _____ / 95.....	44
NIVEL 6 — EL NACIMIENTO DE LOS OBJETOS 🧬	45
Clases con atributos e instancias	45
🧠 CARGA RÁPIDA.....	45
El problema (el que sufriste en el Nivel 5)	45
Paso 1 — Definir la clase (el molde).....	45
Paso 2 — Crear una instancia (la galleta).....	45
Cómo se ve en la memoria.....	46
El operador punto	46
Varias instancias del mismo molde: cada una es independiente.....	47
Valores por defecto	47
💀 LAS DOS TRAMPAS DE ESTE NIVEL.....	47
👁️ TE LO MUESTRO — El gremio de dos héroes	48
👉 LO HACEMOS JUNTOS — La clase Enemigo (10 XP).....	50
💪 AHORA TÚ — La ficha completa del personaje (20 + 15 + 15 XP)	51

 CAZA DE BUGS #6 (25 XP)	51
 ESTACIÓN IA — Nivel 6 · EL FILTRO	52
 CHECKPOINT NIVEL 6 — XP: _____ / 95.....	52
NIVEL 7 — EL EJÉRCITO 	53
Arrays de objetos	53
 CARGA RÁPIDA	53
Acceder a un atributo dentro del array	53
Recorrer un array de objetos	54
 TE LO MUESTRO — El escuadrón completo	54
 LO HACEMOS JUNTOS — La patrulla (10 XP)	56
 AHORA TÚ — El registro del gremio completo (20 + 15 + 15 XP)	56
 CAZA DE BUGS #7 (25 XP)	57
 ESTACIÓN IA — Nivel 7 · EL SABOTEADOR	57
 CHECKPOINT NIVEL 7 — XP: _____ / 95.....	58
 JEFE FINAL — "CÓDIGO DRAGÓN v1.0"	58
Proyecto integrador	58
La situación	58
Requisitos del sistema	58
Restricciones (aquí está la nota del proyecto)	59
Entregables	59
 La regla de oro del jefe final.....	59
 ANEXOS	60
ANEXO A — Chuleta de Java	60
ANEXO B — Diccionario del compilador (traduce el error real).....	61
ANEXO C — Los 14 errores más frecuentes	62
ANEXO D — Soluciones	63
Nivel 0 — La bolsa de oro	63
Nivel 1 — La tienda del herrero	63
Nivel 1 — Caza de bugs #1	63
Nivel 2 — El portero de la mazmorra.....	64
Nivel 2 — Caza de bugs #2	64
Nivel 3 — El entrenamiento.....	64
Nivel 3 — Caza de bugs #3	64
Nivel 4 — Los daños del combate	65
Nivel 4 — Caza de bugs #4	65
Nivel 5 — El daño máximo	65
Nivel 5 — Caza de bugs #5	65

Nivel 6 — La clase Enemigo	65
Nivel 6 — Caza de bugs #6	66
Nivel 7 — La patrulla	66
Nivel 7 — Caza de bugs #7	66
ANEXO E — Rúbrica de evaluación	66
ANEXO F — Hoja de Ruta (tu tablero de XP).....	67
ANEXO G — Bitácora de IA	68
ANEXO H — Guía para el docente.....	¡Error! Marcador no definido.
Fundamentación pedagógica	¡Error! Marcador no definido.
Secuencia y cronograma	¡Error! Marcador no definido.
Sugerencias de implementación	¡Error! Marcador no definido.
Adaptaciones	¡Error! Marcador no definido.
 CIERRE	¡Error! Marcador no definido.

Taller 100% guiado de Fundamentos de Java

Nivel: Primer semestre de universidad ·

Duración: 5 horas (3 sesiones) o 1 jornada maratón

Temas: entrada de datos con Scanner · decisiones · ciclos · arrays · clases con atributos e instancias

Prerrequisitos: saber qué es un algoritmo, un condicional y un ciclo (si hiciste el taller de pseudocódigo, estás listo)

Lo que NO veremos aquí: constructores, métodos propios, herencia. Eso viene después. Aquí construimos la base.

LA HISTORIA (léela, es parte del taller)

Un estudio de videojuegos indie te contrata. El proyecto se llama **Código Dragón**: un RPG por turnos.

El problema es que el programador anterior se fue y dejó el proyecto en cero. Solo dejó un post-it pegado al monitor:

"Todo videojuego es lo mismo: datos que entran, decisiones que se toman, cosas que se repiten, listas que se recorren y objetos que existen. Si sabes hacer esas cinco cosas, sabes hacer un videojuego. Y sabes programar."

Tú vas a construir ese RPG desde cero. **Nivel por nivel, línea por línea.**

- En el **Nivel 1** vas a crear tu héroe pidiéndole los datos al jugador.
- En el **Nivel 3** tu héroe va a pelear contra un dragón en un combate por turnos.
- En el **Nivel 4** vas a tener un inventario.
- En el **Nivel 6** tu héroe va a dejar de ser un montón de variables sueltas y se va a convertir en un **objeto de verdad**.
- En el **Jefe Final** vas a tener un juego completo, jugable, hecho por ti.

¿QUÉ VAS A SABER HACER AL TERMINAR?

1. **Escribir, compilar y ejecutar** un programa Java desde cero sin copiar plantillas.
2. **Capturar datos** del usuario con **Scanner** sin caer en la trampa del salto de línea.
3. **Tomar decisiones** con **if**, **else if**, **else** y **switch**, comparando números y textos correctamente.
4. **Repetir** con **while**, **do-while** y **for**, y saber cuál elegir en cada caso.
5. **Almacenar y recorrer** colecciones de datos con **arrays**, calculando máximos, promedios y búsquedas.
6. **Diseñar tus propias clases** con atributos, **instanciarlas** con **new** y manejar varios objetos a la vez.
7. **Leer un mensaje de error del compilador** y saber exactamente qué significa.

REGLAS DEL JUEGO

7 niveles + 1 jefe final. Cada nivel tiene siempre los mismos 5 pasos:

Paso	Ícono	Qué pasa	Quién trabaja
1		CARGA RÁPIDA — teoría mínima, máximo 2 minutos	Lees
2		TE LO MUESTRO — código completo, explicado línea por línea	Observas
3		LO HACEMOS JUNTOS — código con huecos para rellenar	Completas
4		AHORA TÚ — el reto real del nivel	Creas
5		CAZA DE BUGS — código roto que debes reparar	Reparas

Sistema de puntos (XP)

Acción	XP
Completar "Lo hacemos juntos"	10 XP
Resolver "Ahora tú" y que compile	20 XP
Que además funcione con los datos de prueba	15 XP
Prueba de escritorio a mano	15 XP
Cazar el bug y explicar POR QUÉ fallaba	25 XP
Bonus creativo ( en cada nivel)	10 XP

Total por nivel: 95 XP · Niveles: 665 XP · Jefe final: 200 XP · GRAN TOTAL: 865 XP

Insignias

Insignia	Cómo se gana
 Aprendiz de Java	300 XP
 Domador de Objetos	500 XP
 Arquitecto del Dragón	650 XP
 Leyenda del Estudio	700+ XP y jefe final aprobado
 Cazador de Bugs	Cazar los 7 bugs de los 7 niveles
 Domador de IA	Detectar al menos 3 errores en respuestas de IA
 Traductor del Compilador	Explicar con tus palabras 10 mensajes de error distintos

PROTOCOLO COPILOTO: usar IA sin volverte inútil

Puedes usar IA en este taller. Quiero que la uses. Pero con reglas.

Tú eres el piloto. La IA es el copiloto.

El copiloto sugiere. El piloto decide, verifica y firma.

Permitido

- Pedir que te **traduzca un error del compilador**: "*¿Qué significa ``cannot find symbol`` en esta línea?*" (uso legítimo y utilísimo).
- Pedir **explicaciones alternativas**: "*Explícame los arrays como si fuera un casillero de gimnasio.*"
- Pedir que **te examine**: "*Hazme 5 preguntas de opción múltiple sobre ciclos y califícame.*"
- Pedir **revisión después** de escribir tu código: "*No lo reescribas. Dime en qué línea está el error y por qué.*"
- Pedirle que **rompa** tu programa: "*¿Qué dato escribiría un usuario para que esto falle?*"

Prohibido

- Entregar código que no puedas **explicar línea por línea**.
- Usar cosas que **no hemos visto** (constructores, métodos propios, `ArrayList`, herencia). Si la IA te los mete, quítalos — y ese es justamente uno de los ejercicios.

La Regla de los 60 Segundos

Intenta el problema **60 segundos con papel** antes de escribir cualquier prompt. Ese minuto es el que fabrica el aprendizaje.

El filtro obligatorio de este taller

La IA fue entrenada con millones de líneas de Java **avanzado**. Cuando le pidas ayuda, casi siempre te va a devolver código con constructores, métodos, `ArrayList` y cosas que aún no ves. Por eso, **todo prompt de este taller debe terminar con esta frase**:

"Usa SOLO: variables, Scanner, if/else, switch, while, do-while, for, arrays y clases con atributos públicos instanciadas con new. NO uses constructores, NO uses métodos propios, NO uses ArrayList, NO uses herencia. Todo el código ejecutable va dentro de main."

Guárdala. La vas a pegar unas 20 veces hoy.

 Bitácora de IA (obligatoria, Anexo G)

#	¿Qué le pedí?	¿Qué respondió?	¿Usó algo prohibido?	¿Qué corregí yo?
---	---------------	-----------------	----------------------	------------------

 KIT DE HERRAMIENTAS

Herramienta	Para qué	Cómo conseguirla
JDK 17 o superior	El compilador y el motor de Java	adoptium.net (gratis)
IntelliJ IDEA Community	Editor recomendado, te subraya los errores	jetbrains.com/idea (gratis)
Alternativa sin instalar nada	Programar desde el navegador	onlinegdb.com/online_java_compiler o replit.com
Lápiz y papel	Pruebas de escritorio. No es opcional.	Ya lo tienes

 **Regla de la casa:** los primeros 10 minutos de cada nivel son **sin computador**. El papel te obliga a pensar; el editor te invita a tantear hasta que algo funcione. Tantear no es programar.

NIVEL 0 — CAMPO DE ENTRENAMIENTO *"Antes de crear un dragón, aprende a decir hola"* 35 minutos · Sin XP (esto es entrenamiento) **CARGA RÁPIDA****La anatomía de un programa Java**

Todo programa Java empieza igual. Esto es lo mínimo que existe:

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hola, mundo de Código Dragón");
    }
}
```

Qué significa cada parte (sin profundizar todavía — algunas cosas te las crearás por ahora, y está bien):

Parte	Qué es	Por ahora entiéndelo como...
<code>public class Main</code>	La clase que envuelve todo	"El archivo. Se llama Main, y el archivo debe llamarse <code>Main.java</code> "
<code>{ }</code>	Un bloque	"Todo lo que está adentro pertenece a lo de afuera"
<code>public static void main(String[] args)</code>	El método principal	" La puerta de entrada. Java siempre empieza a ejecutar aquí"
<code>System.out.println(...)</code>	Imprimir en pantalla	"Escribir algo y bajar de línea"
<code>;</code>	Fin de instrucción	"El punto final de la frase"

🚩 Las 4 leyes que no se negocian en Java:

1. El nombre del archivo debe ser idéntico al nombre de la clase pública. `public class Heroe` → `Heroe.java`. Con la misma mayúscula.
2. Java distingue mayúsculas de minúsculas. `String` ≠ `string`. `Scanner` ≠ `scanner`. `main` ≠ `Main`.
3. Cada instrucción termina en ```.
4. Cada ``` que abres, lo cierras. Cuéntalos.

`println` vs `print`

Instrucción	Qué hace
<code>System.out.println("Hola");</code>	Escribe y baja de línea
<code>System.out.print("Hola");</code>	Escribe y se queda ahí (ideal para pedir datos)

Variables: la caja con nombre y con sabor

En Java, **toda variable debe declarar su tipo antes de existir**. Esto es distinto de otros lenguajes y es una de las cosas que más cuesta al principio.

```
int vida = 100;           // enteros
double oro = 250.75;      // números con decimales
String nombre = "Aria";   // texto (¡String con S mayúscula!)
boolean estaVivo = true;  // verdadero o falso
```

```
char inicial = 'A';           // UN solo carácter, comillas simples
```

Tipo	Guarda	Ejemplo	Ojo con...
int	Enteros	100, -5, 0	No admite decimales
double	Decimales	3.5, 250.75	Se escribe con punto , nunca coma
String	Texto	"Aria"	Comillas dobles y S mayúscula
boolean	true / false	true	Sin comillas
char	Un carácter	'A'	Comillas simples

Operadores

```
int suma = 10 + 3;           // 13
int resta = 10 - 3;          // 7
int mult = 10 * 3;           // 30
int div = 10 / 3;             // 3   ← ¡ATENCIÓN!
int resto = 10 % 3;           // 1   (el residuo)
```

🧠 LA TRAMPA MÁS FAMOSA DE JAVA: la división entera.

10 / 3 da 3, no 3.33. Si divides dos enteros, Java te devuelve un entero y **tira los decimales a la basura sin avisarte**.

Para obtener 3.33 necesitas que al menos uno sea decimal:

double resultado = 10.0 / 3; → 3.333...

Este error no te da mensaje de compilación. Simplemente tu programa da resultados mal, y tú no sabes por qué. Recuérdalo hoy y te ahorras 3 semestres de sufrimiento.

Concatenar texto con +

```
String nombre = "Aria";
int vida = 100;
System.out.println("El héroe " + nombre + " tiene " + vida + " puntos de vida");
```



Cuando el + tiene texto de un lado, **deja de sumar y pega**. Por eso "Vida: " + 5 + 3 imprime Vida: 53 y no Vida: 8. Si quieres que sume primero, usa paréntesis: "Vida: " + (5 + 3).

Comentarios

```
// Comentario de una línea

/* Comentario
```

```
de varias líneas */
```

👁️ TE LO MUESTRO — La ficha de estadísticas

Problema: mostrar en pantalla la ficha de un héroe con nombre, vida, ataque y su poder total (vida + ataque × 2).

```
public class FichaEstadisticas {
    public static void main(String[] args) {

        // 1. Declaramos y damos valor a las variables
        String nombre = "Aria la Veloz";
        int vida = 100;
        int ataque = 25;

        // 2. Calculamos
        int poderTotal = vida + ataque * 2;

        // 3. Mostramos
        System.out.println("=====");
        System.out.println("  FICHA DEL HÉROE");
        System.out.println("=====");
        System.out.println("Nombre : " + nombre);
        System.out.println("Vida   : " + vida);
        System.out.println("Ataque : " + ataque);
        System.out.println("PODER  : " + poderTotal);
    }
}
```

Salida esperada:

```
=====
  FICHA DEL HÉROE
=====
Nombre : Aria la Veloz
Vida   : 100
Ataque : 25
PODER  : 150
```

Prueba de escritorio (simula ser el computador, línea por línea):

Paso	Instrucción	nombre	vida	ataque	poderTotal	Pantalla
1	<code>nombre = "Aria la Veloz"</code>	Aria la Veloz	?	?	?	—
2	<code>vida = 100</code>	Aria la Veloz	100	?	?	—
3	<code>ataque = 25</code>	Aria la Veloz	100	25	?	—
4	<code>poderTotal = 100 + 25*2</code>	Aria la Veloz	100	25	150	—
5	<code>println(...)</code>	—	—	—	—	La ficha

🔑 En el paso 4, `25 * 2` se resuelve **antes** que la suma. La multiplicación tiene mayor precedencia. Si querías `(100+25)*2 = 250`, necesitabas paréntesis. **Ante la duda: paréntesis.**

👉 LO HACEMOS JUNTOS — La bolsa de oro

Completa los espacios:

```
public class BolsaDeOro {
    public static void main(String[] args) {

        _____ nombreObjeto = "Poción de vida";
        _____ precioUnitario = 12.5;
        _____ cantidad = 4;

        double totalGastado = _____ * _____;

        System.out.println("Objeto: " + _____);
        System.out.println("Compraste " + _____ + " unidades");
        System.out.println("Gastaste: " + _____ + " monedas de oro");
    }
}
```

Preguntas de control:

1. ¿Por qué `precioUnitario` no puede ser `int`? _____
2. ¿Qué pasa si escribo `12,5` en vez de `12.5`? _____
3. ¿Qué imprime `"Total: " + 4 + 2`? ¿Y `"Total: " + (4 + 2)`? _____

✅ CHECKPOINT NIVEL 0

- ☐ Compilé y ejecuté mi primer programa Java
- ☐ Sé que el archivo debe llamarse igual que la clase pública
- ☐ Entiendo la trampa de la división entera
- ☐ Sé la diferencia entre `print` y `println`
- ☐ Sé declarar los 5 tipos básicos

Si marcaste 4 o más → **al Nivel 1.**

NIVEL 1 — LA CREACIÓN DEL HÉROE 🧙

Entrada de datos con Scanner

🕒 40 minutos · Máx. 95 XP

🧠 CARGA RÁPIDA

Hasta ahora tus programas hacen siempre lo mismo porque los datos están escritos en el código. Un juego así no sirve: el jugador tiene que poder escribir el nombre de SU héroe.

Para eso existe **Scanner**.

Los 3 pasos del Scanner (siempre los mismos)

```
import java.util.Scanner;           // 1. IMPORTAR (arriba del todo, antes de la clase)

public class Ejemplo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in); // 2. CREAR el scanner

        System.out.print("Escribe tu nombre: ");
        String nombre = sc.nextLine();        // 3. LEER

        System.out.println("Hola " + nombre);

        sc.close();                        // 4. Cerrarlo al final (buena práctica)
    }
}
```

🧠 Fíjate en `new Scanner(...)`. Esa palabra `new` significa "créame uno nuevo". Ya la estás usando desde hoy, y en el Nivel 6 vas a entender exactamente qué hace. Guarda la curiosidad.

Los métodos de lectura

Método	Lee	Ejemplo
<code>sc.nextInt()</code>	un número entero	<code>int edad = sc.nextInt();</code>
<code>sc.nextDouble()</code>	un número con decimales	<code>double oro = sc.nextDouble();</code>
<code>sc.nextLine()</code>	una línea completa (admite espacios)	<code>String nom = sc.nextLine();</code>
<code>sc.next()</code>	una sola palabra (corta en el espacio)	<code>String col = sc.next();</code>
<code>sc.nextBoolean()</code>	true / false	<code>boolean b = sc.nextBoolean();</code>

⚠️ `next()` vs `nextLine()`: si el usuario escribe `Aria la Veloz`, `next()` te da solo `"Aria"` y `nextLine()` te da `"Aria la Veloz"`. Para nombres, usa siempre `nextLine()`.

💀 LA TRAMPA DEL SALTO DE LÍNEA (el bug #1 de todo estudiante de Java)

Mira este código. **Parece perfecto. No lo es.**

```
System.out.print("Edad: ");
int edad = sc.nextInt();

System.out.print("Nombre: ");
String nombre = sc.nextLine();    // 💀 ¡ESTA LÍNEA SE SALTA SOLA!
```

Qué pasa realmente: cuando escribes `16` y presionas Enter, quedan **dos cosas** en el buffer del teclado: el número `16` y el **Enter** (el salto de línea).

- `nextInt()` toma el `16`... y **deja el Enter ahí tirado**.
- `nextLine()` llega, encuentra ese Enter suelto, dice *"ah, una línea vacía, listo"* y devuelve `""` sin darte tiempo de escribir nada.

La solución (memorízala): después de un `nextInt()` o `nextDouble()`, si lo siguiente es un `nextLine()`, agrega un `sc.nextLine()` de limpieza.

```
System.out.print("Edad: ");
int edad = sc.nextInt();
sc.nextLine();                // ✅ Limpia el Enter pendiente

System.out.print("Nombre: ");
String nombre = sc.nextLine(); // ahora sí funciona
```

🎯 **Regla de bolsillo:** *número seguido de texto → limpia el Enter*. Texto seguido de texto o número seguido de número, no hay problema.

👁️ TE LO MUESTRO — La creación del héroe

```
import java.util.Scanner;

public class CreacionHeroe {
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

System.out.println("=== CÓDIGO DRAGÓN: creación de personaje ===");

System.out.print("Nombre de tu héroe: ");
String nombre = sc.nextLine();

System.out.print("Edad del héroe: ");
int edad = sc.nextInt();

System.out.print("Vida inicial: ");
int vida = sc.nextInt();

System.out.print("Oro inicial: ");
double oro = sc.nextDouble();

sc.nextLine(); // Limpiamos el Enter antes de Leer texto

System.out.print("Reino de origen: ");
String reino = sc.nextLine();

int poder = vida + edad * 2;

System.out.println();
System.out.println("===== FICHA =====");
System.out.println("Héroe : " + nombre);
System.out.println("Reino : " + reino);
System.out.println("Edad : " + edad);
System.out.println("Vida : " + vida);
System.out.println("Oro : " + oro);
System.out.println("Poder : " + poder);
System.out.println("=====");

sc.close();
}

```

Prueba de escritorio (el usuario escribe: **Aria la Veloz**, **16**, **100**, **250.5**, **Valle del Alba**):

Paso	Instrucción	Qué queda en el buffer	Variable resultante
1	<code>nextLine()</code>	vacío	nombre = "Aria la Veloz"
2	<code>nextInt()</code>	🚩 pendiente	edad = 16
3	<code>nextInt()</code>	🚩 pendiente	vida = 100
4	<code>nextDouble()</code>	🚩 pendiente	oro = 250.5
5	<code>nextLine()</code> (limpieza)	vacío ✅	(se descarta)
6	<code>nextLine()</code>	vacío	reino = "Valle del Alba"

Poder = $100 + 16 \times 2 = 132$

👉 LO HACEMOS JUNTOS — La tienda del herrero (10 XP)

```
import java.util._____;

public class TiendaHerrero {
    public static void main(String[] args) {

        Scanner sc = new _____(System.in);

        System.out.print("¿Cuántas monedas de oro tienes? ");
        double oroDisponible = sc._____;

        System.out.print("Precio de la espada: ");
        double precioEspada = sc._____;

        System.out.print("¿Cuántas espadas quieres? ");
        int cantidad = sc._____;

        _____; // ← ¿hace falta limpiar aquí? Piénsalo

        System.out.print("Nombre del arma personalizada: ");
        String nombreArma = sc._____;

        double costoTotal = _____ * _____;
        double oroRestante = _____ - _____;

        System.out.println("Compraste " + cantidad + " x " + nombreArma);
        System.out.println("Gastaste: " + _____);
        System.out.println("Te quedan: " + _____ + " de oro");

        sc._____;

    }
}
```

Pregunta clave: ¿en qué línea exacta hacía falta el `sc.nextLine()` de limpieza y por qué justo ahí? _____

👉 AHORA TÚ — El registro del gremio (20 + 15 + 15 XP)

Misión 1.1: Escribe un programa que registre a un aventurero en el gremio pidiendo:

1. Nombre completo (con espacios)
2. Edad
3. Vida base
4. Ataque base
5. Defensa base
6. Oro inicial (con decimales)
7. Clase del personaje (una sola palabra: Guerrero, Mago o Arquero)

Y luego muestre una ficha formateada que además calcule:

- **Poder de combate** = ataque × 3 + defensa × 2
- **Nivel estimado** = vida / 10 (entero)

- Oro tras pagar la inscripción de 50 monedas

Pistas guiadas:

-  Pista 1: el nombre lleva espacios → `nextLine()`. La clase es una sola palabra → puedes usar `next()`.
-  Pista 2: entre el último número y el texto, **limpia el Enter**.
-  Pista 3: `vida / 10` con `vida` entero da un entero. Aquí eso está bien y es intencional. Piensa por qué.

Datos de prueba: Kael Tormenta de Hierro, 17, 120, 18, 9, 300.0, Guerrero

Resultado esperado: Poder = 72 · Nivel = 12 · Oro restante = 250.0

 **BONUS CREATIVO (10 XP):** haz que el programa dibuje la ficha con un marco de asteriscos y muestre la vida como una barra: `[|||||||||] 120`

CAZA DE BUGS #1 (25 XP)

Este programa debería pedir el nivel y luego el nombre del arma. **Nunca deja escribir el nombre del arma.**

```
import java.util.Scanner;

public class BugScanner {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nivel del héroe: ");
        int nivel = sc.nextInt();

        System.out.print("Nombre del arma: ");
        String arma = sc.nextLine();

        System.out.println("Nivel " + nivel + " empuñando: " + arma);
        sc.close();
    }
}
```

1. Ejecútalo. ¿Qué imprime en `arma`? _____
2. ¿Por qué el programa "se salta" la pregunta? Explícalo con la palabra **buffer**: _____
3. Escribe la línea que falta y **dónde exactamente** va: _____
4. Reto extra: si en vez de `nextLine()` se usara `next()`, ¿el programa funcionaría? Pruébalo y explica el resultado.

ESTACIÓN IA — Nivel 1

1. Pídele: "Explícame con una metáfora qué es el buffer del teclado y por qué `nextInt()` deja un Enter pendiente." Compara su metáfora con la de este documento y quédate con la que más te sirva.
2. Ahora el reto: pídele que resuelva la **Misión 1.1** (recuerda pegar el filtro obligatorio del Protocolo Copiloto).
3. Audítala con esta lista:

- ☐ ¿Usó `nextLine()` para el nombre completo o usó `next()` y le cortó el apellido?
- ☐ ¿Puso el `sc.nextLine()` de limpieza en el lugar correcto?
- ☐ ¿Usó algo que NO hemos visto (constructores, métodos, `ArrayList`)? Si sí, **bórralo y reescríbelo tú.**
- ☐ ¿Cerró el Scanner?

 Bitácora obligatoria. **Cada error real que encuentres vale 25 XP.**

✓ CHECKPOINT NIVEL 1 — XP: _____ / 95

- ☐ Sé los 4 pasos del Scanner (importar, crear, leer, cerrar)
- ☐ Sé cuándo usar `nextLine()` y cuándo `next()`
- ☐ Domino la trampa del salto de línea y sé explicarla
- ☐ Mi programa pide datos y responde con ellos

NIVEL 2 — EL CAMINO SE BIFURCA

Decisiones: if, else if, else y switch

 45 minutos · Máx. 95 XP

CARGA RÁPIDA

if simple y if / else

```
if (vida <= 0) {
    System.out.println("Has muerto.");
}

if (oro >= precio) {
    System.out.println("Compra realizada");
} else {
    System.out.println("Oro insuficiente");
}
```

Siempre se ejecuta uno de los dos caminos. Nunca los dos. Nunca ninguno.

La escalera: if / else if / else

Para rangos excluyentes:

```
if (vida > 80) {
    System.out.println("Estado: EXCELENTE");
} else if (vida > 50) {
    System.out.println("Estado: HERIDO");
} else if (vida > 0) {
    System.out.println("Estado: CRÍTICO");
} else {
    System.out.println("Estado: DERROTADO");
}
```

🔗 El truco que ahorra la mitad del trabajo: si vas en orden, **no repitas el límite del otro lado**. Cuando llegas a `else if (vida > 50)`, Java ya sabe que la vida NO es mayor que 80. Escribir `vida > 50 && vida <= 80` es redundante. Menos código = menos bugs.

Operadores de comparación y lógicos

Operador	Significa	Ejemplo
<code>==</code>	igual a	<code>vida == 0</code>
<code>!=</code>	distinto de	<code>clase != "Mago"</code>
<code>> < >= <=</code>	mayor, menor, mayor o igual, menor o igual	<code>edad >= 16</code>
<code>&&</code>	Y — ambas verdaderas	<code>vida > 0 && oro >= 50</code>
<code>\</code>	<code>\</code>	<code>`</code>
<code>!</code>	NO — invierte	<code>!estaVivo</code>

🚨 Error clásico #1: ``=`` no es `==``.

`=` **asigna** (guarda un valor). `==` **pregunta** (compara).

`if (vida = 0)` no compila en Java. Y menos mal, porque en otros lenguajes sí compila y arruina programas enteros.

🚨 Error clásico #2: el punto y coma asesino.

```
```java
if (vida > 0); // ← ese ; termina el if AQUÍ MISMO
{
 System.out.println("Vivo"); // esto SIEMPRE se ejecuta
}
```

```
}

```

El compilador **no te avisa**, porque es código legal. Simplemente tu **if** deja de funcionar. **Nunca pongas `` después de ``if`, ``while` o ``for`.**



### LA TRAMPA MÁS IMPORTANTE DE ESTE NIVEL: comparar Strings

```
String clase = sc.nextLine();

if (clase == "Mago") { // 🦴 ESTO ESTÁ MAL Y NO DA ERROR DE COMPILACIÓN
 ...
}
```

**Por qué falla:** en Java, **==** con objetos compara **si son el mismo objeto en memoria**, no si dicen lo mismo. Dos textos idénticos escritos en momentos distintos son objetos distintos, así que **==** da **false** aunque las letras coincidan.

**La forma correcta:**

```
if (clase.equals("Mago")) { ... } // sensible a mayúsculas
if (clase.equalsIgnoreCase("mago")) { ... } // ignora mayúsculas ← recomendado para menús
```



**Regla de oro:** números con **==**, textos con **.equals()**. Sin excepciones. Este error hace que menús enteros "no funcionen" sin ningún mensaje de error, y es responsable de más lágrimas de primer semestre que cualquier otra cosa.

### switch: para valores exactos

```
int opcion = sc.nextInt();

switch (opcion) {
 case 1:
 System.out.println("Guerrero: mucha vida, ataque medio");
 break;
 case 2:
 System.out.println("Mago: poca vida, ataque brutal");
 break;
 case 3:
 System.out.println("Arquero: equilibrado");
 break;
 default:
 System.out.println("Clase no válida");
}
```

⚠ Si olvidas el `break`, Java sigue ejecutando los casos siguientes. Se llama *fall-through* y hace que elegir la opción 1 imprima las tres. Es un bug clásico, y lo vas a cazar hoy.

Usa <code>if / else if</code> cuando...	Usa <code>switch</code> cuando...
comparas rangos ( <code>vida &gt; 50</code> )	comparas valores exactos ( <code>opcion == 3</code> )
hay condiciones compuestas ( <code>&amp;&amp;</code> , <code>\</code> )	<code>\</code>

## 👁 TE LO MUESTRO — El selector de clase y el golpe crítico

```
import java.util.Scanner;

public class SelectorClase {
 public static void main(String[] args) {

 Scanner sc = new Scanner(System.in);

 System.out.println("=== ELIGE TU CLASE ===");
 System.out.println("1. Guerrero 2. Mago 3. Arquero");
 System.out.print("Opción: ");
 int opcion = sc.nextInt();

 int vida = 0;
 int ataque = 0;
 String nombreClase = "";

 switch (opcion) {
 case 1:
 nombreClase = "Guerrero";
 vida = 150;
 ataque = 15;
 break;
 case 2:
 nombreClase = "Mago";
 vida = 80;
 ataque = 35;
 break;
 case 3:
 nombreClase = "Arquero";
 vida = 110;
 ataque = 25;
 break;
 default:
 nombreClase = "Aldeano";
 vida = 50;
 ataque = 5;
 }

 System.out.println("Eres un " + nombreClase + " (vida " + vida + ", ataque " +
 ataque + ")");

 // Ahora, el golpe crítico
 System.out.print("Tira el dado (1 a 20): ");
 int dado = sc.nextInt();

 int danio;
```

```

 if (dato == 20) {
 danio = ataque * 3;
 System.out.println("¡CRÍTICO PERFECTO! Daño x3");
 } else if (dato >= 15) {
 danio = ataque * 2;
 System.out.println("¡Golpe crítico! Daño x2");
 } else if (dato >= 5) {
 danio = ataque;
 System.out.println("Golpe normal");
 } else {
 danio = 0;
 System.out.println("¡Fallaste el ataque!");
 }

 System.out.println("Daño infligido: " + danio);

 sc.close();
}
}

```

Prueba de escritorio con 5 casos (fíjate en los **bordes**):

Caso	opcion	dado	nombreClase	ataque	Rama que entra	danio
A	1	20	Guerrero	15	<code>dado == 20</code>	45
B (borde)	2	15	Mago	35	<code>dado &gt;= 15</code>	70
C (borde)	2	14	Mago	35	<code>dado &gt;= 5</code>	35
D (borde)	3	5	Arquero	25	<code>dado &gt;= 5</code>	25
E	3	4	Arquero	25	<code>else</code>	0
F	9	10	Aldeano	5	<code>dado &gt;= 5</code>	5

🎯 Los casos borde (B, C, D) son los que valen. Prueba siempre el valor exacto del límite y el valor justo por debajo. Ahí vive el 90 % de los bugs del mundo real.

## 👉 LO HACEMOS JUNTOS — El portero de la mazmorra (10 XP)

Solo entra quien tenga **nivel 10 o más** Y esté **vivo**. Si es del gremio, entra con nivel 8.

```

import java.util.Scanner;

public class PorteroMazmorra {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);

 System.out.print("Nivel del héroe: ");
 }
}

```

```

int nivel = sc.nextInt();

System.out.print("Vida actual: ");
int vida = sc.nextInt();
sc.nextLine();

System.out.print("¿Pertenece al gremio? (si/no): ");
String gremio = sc.nextLine();

int nivelMinimo;

if (gremio._____("si")) { // ¿== o .equals? ¿ignora mayúsculas?
 nivelMinimo = ____;
} else {
 nivelMinimo = ____;
}

if (nivel ____ nivelMinimo ____ vida ____ 0) {
 System.out.println("Puedes entrar a la mazmorra");
} else {
 System.out.println("ACCESO DENEGADO");

 if (nivel ____ nivelMinimo) {
 System.out.println("Motivo: nivel insuficiente");
 }
 if (vida ____ 0) {
 System.out.println("Motivo: estás derrotado");
 }
}
sc.close();
}
}

```

**Preguntas:** ¿por qué se usó `equalsIgnoreCase` y no `==`? \_\_\_\_ ¿Qué pasa si el jugador escribe "Si" con mayúscula? \_\_\_\_

## AHORA TÚ — La tienda de pociones (20 + 15 + 15 XP)

**Misión 2.1:** Programa la tienda del pueblo. El jugador tiene una cantidad de oro y elige un producto de un menú:

Opción	Producto	Precio	Efecto
1	Poción pequeña	20	+30 vida
2	Poción grande	50	+80 vida
3	Elixir de fuerza	120	+10 ataque
4	Salir	0	—

El programa debe:

1. Pedir el oro y la vida actuales del héroe (vida máxima = 150).
2. Mostrar el menú y leer la opción con `switch`.
3. **Verificar si tiene oro suficiente.** Si no, mostrar cuánto le falta y no cobrar nada.

4. Si compra, descontar el oro y aplicar el efecto.
5. **La vida nunca puede pasar de 150.** Si se pasa, dejarla exactamente en 150 y avisar "Vida al máximo".
6. Mostrar el estado final.

#### Pistas guiadas:

-  Pista 1: usa `switch` para saber precio y efecto, y `if después` para verificar el oro. Separar "qué elegiste" de "puedes pagarlo" hace el código mucho más limpio.
-  Pista 2: el tope de vida es un `if` de una sola línea: `if (vida > 150) { vida = 150; }`
-  Pista 3: no olvides el `break` en cada `case`.

**Datos de prueba:** oro = 60, vida = 140, opción = 2 → debe comprar, quedar con **10 de oro** y vida **150** (no 220), avisando que quedó al máximo.

 **BONUS CREATIVO (10 XP):** agrega la opción 5, "Poción misteriosa" (30 de oro): si el oro restante es par, cura 100; si es impar, quita 20 de vida. Usa el operador `%`.

## CAZA DE BUGS #2 (25 XP)

Este programa tiene **tres** bugs. Los tres son los clásicos de este nivel.

```
import java.util.Scanner;

public class BugDecisiones {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);

 System.out.print("Escribe tu clase (guerrero/mago): ");
 String clase = sc.nextLine();

 System.out.print("Nivel: ");
 int nivel = sc.nextInt();

 if (clase == "guerrero") {
 System.out.println("Tienes armadura pesada");
 }

 if (nivel >= 10);
 {
 System.out.println("Puedes usar magia avanzada");
 }

 switch (nivel) {
 case 1:
 System.out.println("Novato");
 case 2:
 System.out.println("Aprendiz");
 case 3:
 System.out.println("Veterano");
 }
 sc.close();
 }
}
```

1. Bug 1 (línea del `if (clase == ...)`): ¿por qué nunca entra aunque escribas exactamente `guerrero`? \_\_\_\_\_  
Corrígelos: \_\_\_\_\_
2. Bug 2 (línea del `if (nivel >= 10)`): ¿por qué el mensaje sale siempre, incluso con nivel 1? \_\_\_\_\_
3. Bug 3 (el `switch`): con `nivel = 1`, ¿cuántos mensajes se imprimen? ¿Por qué? \_\_\_\_\_ Corrígelos: \_\_\_\_\_

## ESTACIÓN IA — Nivel 2 · DUELO DE CASOS BORDE

En parejas.

1. Pídele a la IA: *"Escribe un programa Java que clasifique el estado de un héroe: más de 80 de vida es EXCELENTE, más de 50 es HERIDO, más de 0 es CRÍTICO, y 0 o menos es DERROTADO."* (Pega el filtro obligatorio.)
2. **No leas su explicación.** Ve directo a las condiciones.
3. Prueba mentalmente con **vida = 80**, **vida = 50** y **vida = 0** exactos. ¿En qué categoría cae cada uno? ¿Coincide con el enunciado?
4. Ahora pídele: *"¿Qué pasa si la vida es exactamente 80?"* y compara su respuesta con lo que tú comprobaste.
5. Último reto: pregúntale *"¿por qué no debo usar == para comparar Strings en Java?"* y verifica que su explicación mencione la diferencia entre **comparar referencias** y **comparar contenido**. Si no lo menciona, su explicación está incompleta.

 Bitácora. Cada error real = 25 XP.

 **La lección:** la IA casi nunca falla en la sintaxis. Falla en los **bordes** y en lo que **no le dijiste**. Y lo que no le dijiste es responsabilidad tuya.

## CHECKPOINT NIVEL 2 — XP: \_\_\_\_\_ / 95

- ☐ Comparo textos con `.equals()` y números con `==`
- ☐ Nunca pongo `;` después de un `if`
- ☐ Pongo `break` en todos los `case`
- ☐ Pruebo siempre el caso borde

## NIVEL 3 — COMBATE POR TURNOS

Ciclos: **while**, **do-while** y **for**

 45 minutos · Máx. 95 XP

### CARGA RÁPIDA

Un ciclo repite instrucciones. Sin ciclos, un combate de 20 turnos serían 20 bloques de código copiados y pegados.

**while** — *"mientras se cumpla, repite"*

```
int turno = 1; // 1. INICIALIZACIÓN
while (turno <= 5) { // 2. CONDICIÓN
 System.out.println("Turno " + turno);
 turno++; // 3. ACTUALIZACIÓN ← ¡si falta, ciclo infinito!
}
```

Las 3 partes que TODO ciclo necesita:

Parte	Si falta...
1. Inicialización	la variable no existe o vale basura
2. Condición	no compila
3. Actualización	ciclo infinito 🤖 el programa se congela

⚠️ **while** puede ejecutarse **cero veces**. Si la condición es falsa desde el principio, el cuerpo nunca corre. Eso es una característica, no un error.

**do-while** — *"hazlo al menos una vez, luego revisa"*

```
int opcion;
do {
 System.out.println("1. Atacar 2. Huir");
 opcion = sc.nextInt();
} while (opcion != 1 && opcion != 2);
```

	<code>while</code>	<code>do-while</code>
Evalúa la condición	antes	después
Mínimo de ejecuciones	0	1 (siempre)
Ideal para	procesar mientras haya datos	menús y validaciones

⚠ El `do-while` lleva **punto y coma al final**: `} while (cond);` Es el único ciclo que lo lleva.

### for — "repite un número conocido de veces"

```
for (int i = 1; i <= 5; i++) {
 System.out.println("Ataque número " + i);
}
```

//    ↑ inicio    ↑ condición    ↑ actualización — las 3 partes en una sola línea

💡 Por eso `for` es más seguro: **es imposible olvidar la actualización**. Si sabes cuántas veces vas a repetir, usa `for` siempre.

### Contadores, acumuladores y atajos

```
int contador = 0;
contador++; // suma 1 (igual a: contador = contador + 1)
contador--; // resta 1

int total = 0;
total += 15; // suma 15 (igual a: total = total + 15)
total -= 5; // resta 5
```

	Contador	Acumulador
Cuenta <b>cuántos</b>	<code>cont++</code>	—
Suma <b>cuánto</b>	—	<code>suma += valor</code>
Inicia en	0	0

### break y continue (úsalos poco)

```
break; // sale del ciclo INMEDIATAMENTE
continue; // salta a la siguiente vuelta sin terminar esta
```



Son útiles, pero abusar de ellos hace el código difícil de seguir. Regla: si puedes resolverlo con la condición del ciclo, hazlo con la condición.

## 👁️ TE LO MUESTRO — El combate contra el dragón

```
import java.util.Scanner;

public class CombateDragon {
 public static void main(String[] args) {

 Scanner sc = new Scanner(System.in);

 int vidaHéroe = 100;
 int vidaDragon = 120;
 int ataqueHéroe = 25;
 int ataqueDragon = 18;

 int turno = 1;
 int danioTotalHecho = 0;

 System.out.println("=== ¡COMIENZA EL COMBATE! ===");

 while (vidaHéroe > 0 && vidaDragon > 0) {

 System.out.println();
 System.out.println("--- TURNO " + turno + " ---");

 // EL héroe ataca
 vidaDragon -= ataqueHéroe;
 danioTotalHecho += ataqueHéroe;
 System.out.println("Golpeas al dragón por " + ataqueHéroe + ". Le quedan " +
vidaDragon);

 // Si el dragón sigue vivo, contraataca
 if (vidaDragon > 0) {
 vidaHéroe -= ataqueDragon;
 System.out.println("El dragón te quema por " + ataqueDragon + ". Te quedan " +
vidaHéroe);
 }

 turno++;
 }

 System.out.println();
 System.out.println("=== FIN DEL COMBATE ===");

 if (vidaHéroe > 0) {
 System.out.println("¡VICTORIA! Vida restante: " + vidaHéroe);
 } else {
 System.out.println("DERROTA. El dragón te venció.");
 }

 System.out.println("Turnos jugados: " + (turno - 1));
 System.out.println("Daño total infligido: " + danioTotalHecho);

 sc.close();
 }
}
```

**Prueba de escritorio completa** (esto es lo que separa entender de adivinar):

Turno	vidaDragon al inicio	Héroe golpea -25	¿Dragón vivo?	Héroe recibe -18	vidaHéroe
1	120	95	Sí	Sí	82
2	95	70	Sí	Sí	64
3	70	45	Sí	Sí	46
4	45	20	Sí	Sí	28
5	20	-5	No	No contraataca	28

Salí del ciclo porque `vidaDragon > 0` es falso. **Victoria con 28 de vida, en 5 turnos, 125 de daño total.** ✅



**Fíjate en el `if (vidaDragon > 0)` dentro del ciclo.** Sin él, un dragón muerto alcanzaría a darte un último golpe desde el más allá. Detalles como ese son los que hacen que un juego se sienta justo.

## 👉 LO HACEMOS JUNTOS — El entrenamiento (10 XP)

El héroe entrena: gana 5 de experiencia por sesión hasta llegar a 100.

```
public class Entrenamiento {
 public static void main(String[] args) {

 int experiencia = ____; // inicialización
 int sesiones = ____;

 while (experiencia ____ 100) { // condición
 experiencia ____ 5; // actualización del acumulador
 sesiones ____; // contador
 System.out.println("Sesión " + sesiones + " → XP: " + experiencia);
 }

 System.out.println("Entrenamiento completo en " + ____ + " sesiones");
 }
}
```

**Preguntas:**

1. ¿Cuántas sesiones se necesitan? \_\_\_\_\_
2. ¿Qué pasa si borras la línea `experiencia += 5;`? \_\_\_\_\_
3. Reescríbelo usando `for`. ¿Cuál de las dos versiones te parece más clara y por qué? \_\_\_\_\_

## 👉 AHORA TÚ — El torneo del gremio (20 + 15 + 15 XP)

**Misión 3.1:** Programa un torneo donde el jugador pelea rondas seguidas.

Requisitos:

1. El héroe empieza con 100 de vida y 0 de oro.
2. Se usa un ``do-while`` con este menú, que se repite hasta que el jugador elija salir o muera:

1. Pelear	2. Descansar	3. Ver estado	4. Retirarse
-----------	--------------	---------------	--------------

3. **Pelear:** el jugador pierde 15 de vida y gana 40 de oro. Lleva un contador de peleas ganadas.
4. **Descansar:** recupera 20 de vida (tope 100) pero cuesta 10 de oro. Si no tiene oro, no puede descansar.
5. **Ver estado:** muestra vida, oro y peleas ganadas.
6. Si la vida llega a 0 o menos, el torneo termina con `"Has sido eliminado del torneo"`.
7. Al salir, mostrar un resumen: peleas ganadas, oro final y vida final.

**Pistas guiadas:**

- 🔑 Pista 1: las inicializaciones (`vida`, `oro`, `peleas`) van **antes** del `do`. Si las pones adentro, se reinician en cada vuelta y nunca acumulas nada. Este es el error que comete casi todo el mundo aquí.
- 🔑 Pista 2: la condición de salida tiene **dos partes**: `while (opcion != 4 && vida > 0);`
- 🔑 Pista 3: constrúyelo por pedazos. Primero el menú vacío que solo repite. Luego una opción. Luego otra. Prueba cada una antes de seguir. Eso se llama **desarrollo incremental** y es como se programa en la vida real.

**Secuencia de prueba:** `1 → 1 → 3 → 2 → 1 → 4`

🌶️ **BONUS CREATIVO (10 XP):** que cada 3 peleas ganadas aparezca un jefe que quita 40 de vida pero da 150 de oro. Usa el operador `%`.

## 🐛 CAZA DE BUGS #3 (25 XP)

Este programa debía sumar los números del 1 al 10. **Se congela para siempre.**

```
public class BugCiclo {
 public static void main(String[] args) {
 int i = 1;
 int suma = 0;

 while (i <= 10) {
 suma += i;
 }

 System.out.println("La suma es: " + suma);
 }
}
```

}

1. ¿Cuál de las 3 partes del ciclo falta? \_\_\_\_\_
2. Escribe la línea que falta y **dónde exactamente** va: \_\_\_\_\_
3. Reto extra: si alguien "arregla" el programa poniendo `i++` **antes** de `suma += i`, ¿cuánto daría la suma? Haz la prueba de escritorio y compárala con el resultado correcto (55). \_\_\_\_\_
4. Reto extra 2: reescríbelo con `for` en **tres líneas**. Explica por qué con `for` este bug era imposible. \_\_\_\_\_



### ESTACIÓN IA — Nivel 3 · CAZA DEL CICLO INFINITO

1. Pídele: "Escribe un programa Java con un ciclo `while` que tenga un bug de ciclo infinito, pero **NO** me digas dónde está."
2. Encuéntralo tú con prueba de escritorio (con simular 4 vueltas basta para detectarlo).
3. Ahora al revés: pídele que resuelva la **Misión 3.1** y audita:
  - ☐ ¿Puso las inicializaciones **antes** del `do` o las metió adentro?
  - ☐ ¿La condición de salida contempla las dos causas (salir y morir)?
  - ☐ ¿El tope de vida en 100 está bien aplicado?
  - ☐ ¿Usó algo prohibido (métodos, constructores)?
4. Pregunta final: "¿Cuándo **NO** se puede convertir un `while` en un `for`?" Contrástalo con la tabla de este nivel.



Bitácora.



### CHECKPOINT NIVEL 3 — XP: \_\_\_\_\_ / 95

- ☐ Sé nombrar las 3 partes de un ciclo
- ☐ Sé cuándo usar `while`, `do-while` y `for`
- ☐ Pongo las inicializaciones ANTES del ciclo
- ☐ Sé usar `++` y `+=`

## NIVEL 4 — EL INVENTARIO

### Arrays: declarar, llenar y recorrer

 45 minutos · Máx. 95 XP

### CARGA RÁPIDA

El problema que resuelven los arrays. Imagina que tu héroe carga 5 pociones:

```
int pocion1 = 30;
int pocion2 = 50;
int pocion3 = 20;
int pocion4 = 80;
int pocion5 = 45; // ¿y si fueran 100 pociones? 🤖
```

Un **array** es una sola variable que guarda **muchos valores del mismo tipo**, numerados.

### Las dos formas de crear un array

```
// FORMA 1: vacío, con tamaño fijo (se llena después)
int[] pociones = new int[5]; // 5 casillas, todas en 0

// FORMA 2: con valores desde el inicio
int[] pociones = {30, 50, 20, 80, 45};
String[] nombres = {"Aria", "Kael", "Nyx"};
```

### Los índices empiezan en CERO

```
int[] pociones = {30, 50, 20, 80, 45};

// índice: 0 1 2 3 4 ← ¡el último es 4, no 5!
```

```
System.out.println(pociones[0]); // 30 ← La PRIMERA
System.out.println(pociones[4]); // 45 ← La ÚLTIMA
System.out.println(pociones[5]); // ✨ ArrayIndexOutOfBoundsException
```

 **Un array de tamaño 5 tiene índices del 0 al 4.** El último índice siempre es `length - 1`. Si intentas leer `pociones[5]`, el programa **explota en tiempo de ejecución** con `ArrayIndexOutOfBoundsException`. No es un error de compilación: el programa arranca bien y se cae en la mitad.

## `.length` — cuántas casillas tiene

```
System.out.println(pociones.length); // 5 ← ¡SIN paréntesis!
```

⚠️ ``length` vs length()`: los **arrays** usan `length` (sin paréntesis). Los **String** usan `length()` (con paréntesis). Sí, es inconsistente. Sí, te vas a equivocar. Ahora ya sabes por qué.

## Recorrer un array

```
// Con for clásico - te da el índice, puedes modificar
for (int i = 0; i < pociones.length; i++) {
 System.out.println("Poción " + i + ": " + pociones[i]);
}

// Con for-each (for mejorado) - más corto, pero SOLO para Leer
for (int p : pociones) {
 System.out.println(p);
}
```

🔴 La condición correcta es ``i < length``, **NUNCA** ``i <= length``.

Con `<=` intentas leer una casilla que no existe y el programa explota. Este es el **error #1 con arrays**, y se llama *off-by-one*.

## Modificar una posición

```
pociones[2] = 99; // La tercera poción ahora vale 99
```

## 👁️ TE LO MUESTRO — El inventario del héroe

```
import java.util.Scanner;

public class Inventario {
 public static void main(String[] args) {

 Scanner sc = new Scanner(System.in);

 int[] pociones = new int[5];
 String[] nombres = {"Poción menor", "Poción media", "Poción mayor", "Elixir", "Antídoto"};

 // 1. LLENAR el array pidiendo datos
 System.out.println("=== CARGA DE INVENTARIO ===");
 for (int i = 0; i < pociones.length; i++) {
```

```

 System.out.print("¿Cuántas unidades de " + nombres[i] + "? ");
 pociones[i] = sc.nextInt();
 }

 // 2. RECORRER para mostrar
 System.out.println();
 System.out.println("=== TU MOCHILA ===");
 for (int i = 0; i < pociones.length; i++) {
 System.out.println("[" + i + "] " + nombres[i] + " x" + pociones[i]);
 }

 // 3. RECORRER para calcular
 int totalObjetos = 0;
 for (int i = 0; i < pociones.length; i++) {
 totalObjetos += pociones[i];
 }

 System.out.println();
 System.out.println("Tipos distintos : " + pociones.length);
 System.out.println("Objetos totales : " + totalObjetos);

 // 4. Modificar una posición concreta
 System.out.print("¿Qué posición quieres vaciar (0 a 4)? ");
 int pos = sc.nextInt();

 if (pos >= 0 && pos < pociones.length) {
 pociones[pos] = 0;
 System.out.println(nombres[pos] + " descartada.");
 } else {
 System.out.println("Esa posición no existe en la mochila.");
 }

 sc.close();
}

```



**Fíjate en el paso 4.** Antes de usar el índice que escribió el usuario, se **valida** que esté entre 0 y `length - 1`.

Sin esa validación, un usuario escribiendo **9** hace explotar tu programa. **Nunca confíes en un índice que venga de afuera.**

**Prueba de escritorio** (el usuario carga **2, 1, 0, 3, 5** y luego vacía la posición 3):

i	nombres[i]	pociones[i]	totalObjetos acumulado
0	Poción menor	2	2
1	Poción media	1	3
2	Poción mayor	0	3
3	Elixir	3	6
4	Antídoto	5	11

Estado final tras vaciar la posición 3: **[2, 1, 0, 0, 5]** 

## 👉 LO HACEMOS JUNTOS — Los daños del combate (10 XP)

```
public class DaniosCombate {
 public static void main(String[] args) {

 int[] danios = {12, 25, 8, 30, 15, 22};

 System.out.println("El array tiene " + danios._____ + " golpes registrados");
 System.out.println("El primer golpe fue de " + danios[_____]);
 System.out.println("El último golpe fue de " + danios[danios._____ - _____]);

 int suma = _____;

 for (int i = _____; i _____ danios.length; i _____) {
 suma _____ danios[i];
 System.out.println("Golpe " + (i + 1) + ": " + danios[_____]);
 }

 System.out.println("Daño total: " + _____);
 }
}
```

### Preguntas:

1. ¿Por qué se imprime `(i + 1)` y no `i` en el mensaje "Golpe N"? \_\_\_\_\_
2. ¿Qué pasaría con `i <= danios.length`? \_\_\_\_\_
3. ¿Cuál es el índice del último elemento? \_\_\_\_\_

## 👉 AHORA TÚ — La bitácora de la mazmorra (20 + 15 + 15 XP)

**Misión 4.1:** El héroe explora una mazmorra de **7 salas**. Escribe un programa que:

1. Pida el oro encontrado en cada una de las 7 salas y lo guarde en un array.
2. Muestre la bitácora completa: **Sala 1: 45 de oro** (numeradas desde 1, aunque el índice empiece en 0).
3. Calcule y muestre el **oro total** encontrado.
4. Pregunte por el número de una sala (1 a 7) y muestre cuánto oro había ahí, **validando** que el número exista.
5. Muestre cuántas salas estaban **vacías** (0 de oro).

### Pistas guiadas:

- 🗝️ Pista 1: si el usuario dice "sala 3", el índice del array es `3 - 1 = 2`. Esa conversión entre "lo que ve el humano" y "lo que usa el array" la vas a hacer toda tu vida.
- 🗝️ Pista 2: para contar las vacías necesitas un **contador** y un `if (oro[i] == 0)` dentro del recorrido.
- 🗝️ Pista 3: valida el índice con `if (sala >= 1 && sala <= 7)` **antes** de usarlo.

**Datos de prueba:** 45, 0, 120, 30, 0, 75, 10 → total = 280, salas vacías = 2

🌶️ **BONUS CREATIVO (10 XP):** dibuja un gráfico de barras en texto, una `#` por cada 10 de oro:

Sala 1 | ####  
Sala 3 | #####

**CAZA DE BUGS #4 (25 XP)**

```
public class BugArray {
 public static void main(String[] args) {

 int[] vidas = {100, 85, 60, 40};

 for (int i = 0; i <= vidas.length; i++) {
 System.out.println("Héroe " + i + ": " + vidas[i]);
 }

 System.out.println("Total de héroes: " + vidas.length());
 }
}
```

1. **Bug 1:** ejecuta el programa. ¿Cuántas líneas alcanza a imprimir antes de caerse? ¿Cuál es el mensaje de error exacto? \_\_\_\_\_
2. ¿Por qué `i <= vidas.length` es incorrecto? Escribe el índice máximo válido: \_\_\_\_\_
3. **Bug 2:** ¿por qué `vidas.length()` no compila? ¿Cuál es la diferencia con `String`? \_\_\_\_\_
4. Corrige ambos y verifica.

**ESTACIÓN IA — Nivel 4 · EL PREDICTOR**

1. Pídele a la IA que genere **un programa Java corto con arrays** (recuerda el filtro).
2. **Antes de ejecutarlo**, predice tú con prueba de escritorio exactamente qué va a imprimir, línea por línea.
3. Ejecútalo. ¿Acertaste tú? ¿Acertó la IA con lo que dijo que haría?
4. Ahora pídele: *"Modifica este código para que tenga un error de índice fuera de rango."* Ejecútalo, **lee el mensaje de excepción completo** y aprende a interpretarlo: te dice el índice que intentaste y el tamaño real (`Index 4 out of bounds for length 4`).



Bitácora.

**CHECKPOINT NIVEL 4 — XP: \_\_\_\_\_ / 95**

- ☐ Sé que los índices van de 0 a `length - 1`
- ☐ Uso `i < length`, nunca `i <= length`
- ☐ Sé la diferencia entre `length` y `length()`
- ☐ Valido los índices que vienen del usuario

## NIVEL 5 — EL SALÓN DE LA FAMA 🏆

### Arrays + ciclos: máximos, búsquedas y estadísticas

🕒 45 minutos · Máx. 95 XP

### 🧠 CARGA RÁPIDA

Ya sabes recorrer un array. Ahora vas a aprender los **4 patrones clásicos** que se repiten en todo programa profesional. Memoriza los cuatro y resuelves el 80 % de los ejercicios de la carrera.

#### Patrón 1 — Suma y promedio

```
int suma = 0;
for (int i = 0; i < puntajes.length; i++) {
 suma += puntajes[i];
}
double promedio = (double) suma / puntajes.length;
```

🔴 Ese `(double)` no está de adorno. Si `suma = 7` y hay 2 elementos, `suma / length` da **3** (división entera). El `(double)` fuerza a Java a trabajar con decimales y da **3.5**. Se llama **casting**. Sin él, todos tus promedios salen mal y el compilador no dice nada.

#### Patrón 2 — Máximo (y su posición)

```
int maximo = puntajes[0]; // el PRIMER elemento es el campeón provisional
int posMaximo = 0;

for (int i = 1; i < puntajes.length; i++) { // empieza en 1, el 0 ya lo tomamos
 if (puntajes[i] > maximo) {
 maximo = puntajes[i];
 posMaximo = i;
 }
}
```

❌ **Nunca inicialices `int maximo = 0`.** Si todos los valores fueran negativos, tu máximo saldría 0, que no es ningún dato real. Es un bug silencioso y elegante que engaña a mucha gente. **El primer elemento siempre es el punto de partida correcto.**

🔑 Y fíjate: para saber **cuál** fue el máximo (no solo cuánto), necesitas guardar también su **posición**.

### Patrón 3 — Búsqueda

```
int buscado = 50;
int posicion = -1; // -1 significa "no encontrado"

for (int i = 0; i < puntajes.length; i++) {
 if (puntajes[i] == buscado) {
 posicion = i;
 break; // ya lo encontramos, no sigas buscando
 }
}

if (posicion != -1) {
 System.out.println("Encontrado en la posición " + posicion);
} else {
 System.out.println("No está en el array");
}
```

 **La bandera** `-1`. Como no existe la posición `-1`, ese valor es una señal segura de "no encontrado". Es una convención universal en programación.

### Patrón 4 — Conteo condicional

```
int cuantosAprobados = 0;
for (int i = 0; i < puntajes.length; i++) {
 if (puntajes[i] >= 60) {
 cuantosAprobados++;
 }
}
```

### Arrays paralelos (y por qué son un problema)

Cuando cada héroe tiene nombre, vida y ataque, la única herramienta que tienes hasta ahora es usar **tres arrays coordinados**:

```
String[] nombres = {"Aria", "Kael", "Nyx"};
int[] vidas = {100, 150, 80};
int[] ataques = {25, 15, 35};

// El héroe i está repartido en tres sitios distintos:
System.out.println(nombres[1] + " tiene " + vidas[1] + " de vida y " + ataques[1] + " de ataque");
```

 **Esto funciona, pero es frágil.** Si ordenas un array y olvidas ordenar los otros, los datos se descoordinan y Kael termina con la vida de Nyx. Si quieres agregar "oro", tienes que tocar cuatro sitios.

**Guarda esta incomodidad.** En el Nivel 6 vas a descubrir la herramienta que la elimina de raíz: las **clases**. Este es exactamente el problema histórico que las hizo nacer.

## 👁️ TE LO MUESTRO — El salón de la fama del gremio

```
import java.util.Scanner;

public class SalonFama {
 public static void main(String[] args) {

 Scanner sc = new Scanner(System.in);

 String[] nombres = {"Aria", "Kael", "Nyx", "Doran", "Selene"};
 int[] puntajes = new int[5];

 // Cargar los puntajes
 for (int i = 0; i < nombres.length; i++) {
 System.out.print("Puntaje de " + nombres[i] + ": ");
 puntajes[i] = sc.nextInt();
 }

 // Patrón 1: suma y promedio
 int suma = 0;
 for (int i = 0; i < puntajes.length; i++) {
 suma += puntajes[i];
 }
 double promedio = (double) suma / puntajes.length;

 // Patrón 2: máximo y mínimo con sus posiciones
 int maximo = puntajes[0];
 int minimo = puntajes[0];
 int posMax = 0;
 int posMin = 0;

 for (int i = 1; i < puntajes.length; i++) {
 if (puntajes[i] > maximo) {
 maximo = puntajes[i];
 posMax = i;
 }
 if (puntajes[i] < minimo) {
 minimo = puntajes[i];
 posMin = i;
 }
 }

 // Patrón 4: conteo condicional
 int sobrePromedio = 0;
 for (int i = 0; i < puntajes.length; i++) {
 if (puntajes[i] > promedio) {
 sobrePromedio++;
 }
 }

 System.out.println();
 System.out.println("===== SALÓN DE LA FAMA =====");
 System.out.println("Promedio del gremio : " + promedio);
 System.out.println("Campeón : " + nombres[posMax] + " con " + maximo);
 System.out.println("Último : " + nombres[posMin] + " con " + minimo);
 System.out.println("Sobre el promedio: " + sobrePromedio + " miembros");

 // Patrón 3: búsqueda por nombre
 }
}
```

```

 sc.nextLine();
 System.out.print("¿A quién buscas? ");
 String buscado = sc.nextLine();

 int posicion = -1;
 for (int i = 0; i < nombres.length; i++) {
 if (nombres[i].equalsIgnoreCase(buscado)) {
 posicion = i;
 break;
 }
 }

 if (posicion != -1) {
 System.out.println(nombres[posicion] + " está en el puesto " + (posicion + 1) + " con "
+ puntajes[posicion] + " puntos");
 } else {
 System.out.println("Ese héroe no pertenece al gremio");
 }

 sc.close();
 }
}

```

Prueba de escritorio (puntajes: 120, 340, 90, 200, 500):

i	puntajes[i]	suma	maximo	posMax	minimo	posMin
0	120	120	120	0	120	0
1	340	460	340	1	120	0
2	90	550	340	1	<b>90</b>	<b>2</b>
3	200	750	340	1	90	2
4	500	1250	<b>500</b>	<b>4</b>	90	2

Promedio =  $1250 / 5 = 250.0$  · Campeón = **Selene (500)** · Sobre el promedio = **2** (Kael y Selene) 

## LO HACEMOS JUNTOS — El daño máximo (10 XP)

```

public class DanioMaximo {
 public static void main(String[] args) {

 int[] golpes = {12, 45, 8, 45, 30, 22};

 int maximo = golpes[____]; // ¿por qué NO 0?
 int posMax = ____;

 for (int i = ____; i < golpes.length; i++) {
 if (golpes[i] ____ maximo) {
 maximo = ____;
 posMax = ____;
 }
 }
 }
}

```

```

 System.out.println("El golpe más fuerte fue de " + _____);
 System.out.println("Ocurrió en el turno " + (_____ + 1));
 }
}

```

**Preguntas:**

1. Hay **dos** golpes de 45 (índices 1 y 3). ¿Cuál de los dos reporta el programa y por qué? \_\_\_\_\_
2. ¿Qué habría que cambiar para que reportara el último en vez del primero? \_\_\_\_\_
3. ¿Por qué el ciclo empieza en **i = 1** y no en **i = 0**? \_\_\_\_\_

## 🔑 AHORA TÚ — El análisis de la temporada (20 + 15 + 15 XP)

**Misión 5.1:** El gremio registra el oro ganado en cada uno de los **12 meses** del año. Escribe un programa que:

1. Pida los 12 valores con un **for**.
2. Muestre el **total anual** y el **promedio mensual** (con decimales correctos — cuidado con el casting).
3. Indique el **mejor mes** (número de mes y valor) y el **peor mes**.
4. Cuente **cuántos meses** estuvieron por encima del promedio.
5. Pida un valor y **busque** si algún mes ganó exactamente ese oro, informando en cuál (o que no ocurrió).
6. Muestre la **diferencia** entre el mejor y el peor mes.

**Pistas guiadas:**

- 🔑 Pista 1: los meses que ve el usuario van de 1 a 12; los índices de 0 a 11. Convierte con **+1** solo al **mostrar**.
- 🔑 Pista 2: para el punto 4 necesitas el promedio, que solo existe **después** de recorrer todo. Por eso necesitas **dos recorridos**: uno para sumar, otro para comparar. Es normal y correcto.
- 🔑 Pista 3: máximo y mínimo pueden calcularse en el **mismo** ciclo. No hacen falta dos.

**Datos de prueba:** 450, 200, 780, 300, 150, 900, 620, 380, 210, 540, 700, 480

**Verificación:** total = **5710** · promedio ≈ **475.83** · mejor = mes 6 (900) · peor = mes 5 (150) · sobre el promedio = 6

🌶️ **BONUS CREATIVO (10 XP):** marca cada mes con un símbolo: ▲ si superó el promedio, ▼ si no.

## 🐛 CAZA DE BUGS #5 (25 XP)

Este programa debía encontrar el máximo y el promedio. Da **los dos mal**.

```

public class BugEstadisticas {
 public static void main(String[] args) {

 int[] temperaturas = {-5, -12, -3, -7};
 }
}

```

```

int maximo = 0;
int suma = 0;

for (int i = 0; i < temperaturas.length; i++) {
 if (temperaturas[i] > maximo) {
 maximo = temperaturas[i];
 }
 suma += temperaturas[i];
}

double promedio = suma / temperaturas.length;

System.out.println("Máximo: " + maximo);
System.out.println("Promedio: " + promedio);
}
}

```

1. **Bug 1:** el máximo imprime **0**, pero 0 no está en el array. ¿Por qué? \_\_\_\_\_ Corrígelo: \_\_\_\_\_
2. **Bug 2:** la suma es **-27** y hay 4 elementos, así que el promedio debería ser **-6.75**. ¿Qué imprime realmente y por qué? \_\_\_\_\_ Corrígelo: \_\_\_\_\_
3. Reto extra: ¿por qué el bug 2 **no se notaría** si los valores fueran **-4, -8, -12, -8**? Explica por qué el hecho de que a veces funcione lo hace todavía más peligroso. \_\_\_\_\_



## ESTACIÓN IA — Nivel 5 · TÚ ERES EL PROFESOR

Cambio de roles: ahora tú calificas.

1. Pídele a la IA que resuelva la **Misión 5.1** (con el filtro obligatorio).
2. Califícala con esta rúbrica:
  - ☐ ¿Inicializó el máximo con el primer elemento o con 0?
  - ☐ ¿Hizo el casting a **double** en el promedio?
  - ☐ ¿Convirtió bien el índice al número de mes al mostrar?
  - ☐ ¿Usó **-1** como bandera de "no encontrado" o algo más frágil?
  - ☐ ¿Usó algo prohibido?
3. Escribe una **retroalimentación de 3 líneas** para la IA, como si fueras el profesor.
4. Envíasela y observa si corrige bien o si "arregla" algo que estaba bien.



Bitácora.



## CHECKPOINT NIVEL 5 — XP: \_\_\_\_\_ / 95

- ☐ Domino los 4 patrones (suma, máximo, búsqueda, conteo)
- ☐ Sé por qué el máximo se inicializa con el primer elemento
- ☐ Hago casting a **double** en los promedios
- ☐ Entiendo por qué los arrays paralelos son incómodos

## NIVEL 6 — EL NACIMIENTO DE LOS OBJETOS

### Clases con atributos e instancias

 55 minutos · Máx. 95 XP

### CARGA RÁPIDA

#### El problema (el que sufriste en el Nivel 5)

Tres arrays paralelos para describir a un héroe. Frágil, disperso, difícil de mantener.

**La solución:** crear tu **propio tipo de dato**. Igual que existen `int` y `String`, tú vas a crear el tipo `Heroe`.

#### Paso 1 — Definir la clase (el molde)

En un archivo nuevo llamado `Heroe.java`:

```
public class Heroe {
 String nombre;
 int vida;
 int ataque;
 double oro;
 boolean estaVivo;
}
```

Eso es todo. Cinco líneas. **Esa clase no hace nada todavía: solo describe qué datos tiene un héroe.** Es un **molde**, no un héroe.



**La metáfora clave:** una clase es el **molde de galletas**; un objeto es la **galleta**. Con un molde haces todas las galletas que quieras, y cada una es independiente: si le muerdes una oreja a la primera, la segunda sigue completa.

#### Paso 2 — Crear una instancia (la galleta)

En tu archivo con `main` (por ejemplo `Juego.java`, en la misma carpeta o proyecto):

```
public class Juego {
 public static void main(String[] args) {

 Heroe h1 = new Heroe(); // ← ¡acabas de crear un objeto!
```

```

 h1.nombre = "Aria";
 h1.vida = 100;
 h1.ataque = 25;
 h1.oro = 250.5;
 h1.estaVivo = true;

 System.out.println(h1.nombre + " tiene " + h1.vida + " de vida");
 }
}

```

Desarmemos la línea más importante de todo el taller:

```

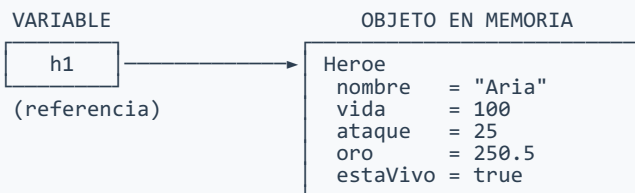
Heroe h1 = new Heroe();
↑ ↑ ↑ ↑
tipo nombre se construye un objeto NUEVO
 de la guarda en la memoria
 variable en

```

- **Heroe h1** → declara una variable **que puede apuntar a** un héroe. Todavía no hay héroe.
- **new Heroe()** → **construye el objeto de verdad** en la memoria, con todos sus atributos.
- **=** → conecta la variable con el objeto.

🧠 ¿Reconoces ese **new**? Lo llevas usando desde el Nivel 1: **new Scanner(System.in)**. Ahora ya sabes qué significaba: *"constrúyeme un objeto nuevo de este tipo"*. Los paréntesis vacíos son el constructor por defecto que Java te regala; **construir tus propios constructores es el tema siguiente, y por eso aquí asignamos los atributos uno por uno con el punto**.

## Cómo se ve en la memoria



🔑 **La variable NO contiene al héroe: contiene la dirección donde vive el héroe.** Por eso se llama **referencia**. Esta idea es la que hace que **==** no funcione con Strings (Nivel 2): compara direcciones, no contenido. Todo encaja.

## El operador punto .

```

h1.vida = 80; // asignar
System.out.println(h1.ataque); // leer

```

```
h1.vida = h1.vida - 20; // usar y modificar
h1.oro += 100; // los atajos también funcionan
```

Léelo como un genitivo: `h1.vida` es "la vida de `h1`".

## Varias instancias del mismo molde: cada una es independiente

```
Heroe h1 = new Heroe();
h1.nombre = "Aria";
h1.vida = 100;

Heroe h2 = new Heroe();
h2.nombre = "Kael";
h2.vida = 150;

h1.vida = 20; // solo afecta a Aria

System.out.println(h1.vida); // 20
System.out.println(h2.vida); // 150 ← Kael ni se enteró
```

## Valores por defecto

Si creas un objeto y no asignas un atributo, Java le pone un valor por defecto:

Tipo	Valor por defecto
<code>int</code>	<code>0</code>
<code>double</code>	<code>0.0</code>
<code>boolean</code>	<code>false</code>
<code>String</code> y cualquier objeto	<code>null</code>



## LAS DOS TRAMPAS DE ESTE NIVEL

### Trampa 1 — Olvidar el ``new``:

```
Heroe h1; // declaraste la variable, pero NO hay objeto
h1.nombre = "Aria"; // ⚡ NullPointerException
```

`h1` no apunta a nada (`null`). Pedirle un atributo a la nada revienta el programa. **Declarar no es crear. Sin ``new``, no hay objeto.**

### Trampa 2 — Dos variables apuntando al mismo objeto (aliasing):

```

Heroe h1 = new Heroe();
h1.vida = 100;

Heroe h2 = h1; // ⚠ NO copia el héroe: copia la DIRECCIÓN

h2.vida = 10;

System.out.println(h1.vida); // 10 ← ¡también cambió!

```



🔗 Para tener **dos héroes de verdad**, necesitas **dos `new`**. Un **new**, un objeto. Cuéntalos.

## 👁 TE LO MUESTRO — El gremio de dos héroes

### Archivo 1 — `Heroe.java`:

```

public class Heroe {
 String nombre;
 String clase;
 int vida;
 int ataque;
 double oro;
 boolean estaVivo;
}

```

### Archivo 2 — `Juego.java`:

```

import java.util.Scanner;

public class Juego {
 public static void main(String[] args) {

 Scanner sc = new Scanner(System.in);

 // ---- HÉROE 1: datos escritos por nosotros ----
 Heroe h1 = new Heroe();
 h1.nombre = "Aria la Veloz";
 h1.clase = "Arquero";
 h1.vida = 110;
 h1.ataque = 25;
 h1.oro = 300.0;
 h1.estaVivo = true;

 // ---- HÉROE 2: datos pedidos al jugador ----
 Heroe h2 = new Heroe();
 }
}

```

```

 System.out.print("Nombre de tu héroe: ");
 h2.nombre = sc.nextLine();

 System.out.print("Clase: ");
 h2.clase = sc.nextLine();

 System.out.print("Vida: ");
 h2.vida = sc.nextInt();

 System.out.print("Ataque: ");
 h2.ataque = sc.nextInt();

 System.out.print("Oro: ");
 h2.oro = sc.nextDouble();

 h2.estaVivo = true;

 // ---- MOSTRAR ----
 System.out.println();
 System.out.println("==== GREMIO =====");
 System.out.println(h1.nombre + " (" + h1.clase + ") - vida " + h1.vida + ", ataque " +
h1.ataque);
 System.out.println(h2.nombre + " (" + h2.clase + ") - vida " + h2.vida + ", ataque " +
h2.ataque);

 // ---- COMPARAR ----
 if (h1.ataque > h2.ataque) {
 System.out.println("El más fuerte es " + h1.nombre);
 } else if (h2.ataque > h1.ataque) {
 System.out.println("El más fuerte es " + h2.nombre);
 } else {
 System.out.println("Están empatados en ataque");
 }

 // ---- MODIFICAR UN OBJETO ----
 System.out.println();
 System.out.println(h2.nombre + " recibe un golpe de " + h1.ataque);
 h2.vida -= h1.ataque;

 if (h2.vida <= 0) {
 h2.vida = 0;
 h2.estaVivo = false;
 System.out.println(h2.nombre + " ha caído.");
 } else {
 System.out.println("Le quedan " + h2.vida + " de vida");
 }

 System.out.println("Vida de " + h1.nombre + ": " + h1.vida + " (intacta, es otro objeto)");

 sc.close();
 }
}

```

**Prueba de escritorio** (el jugador crea a **Kael**, **Guerrero**, **60**, **18**, **100.0**):

Paso	Instrucción	h1.vida	h2.vida	h2.estaVivo
1	<b>new Heroe()</b> para h1	0 (defecto)	—	—
2	asignaciones de h1	110	—	—
3	<b>new Heroe()</b> para h2	110	0 (defecto)	false (defecto)

Paso	Instrucción	h1.vida	h2.vida	h2.estaVivo
4	asignaciones de h2	110	60	true
5	<code>h2.vida -= 25</code>	110	<b>35</b>	true
6	<code>if (h2.vida &lt;= 0) → falso</code>	110	35	true

**h1 nunca se tocó.** Dos objetos, dos vidas separadas. 

## LO HACEMOS JUNTOS — La clase Enemigo (10 XP)

Archivo `Enemigo.java`:

```
public _____ Enemigo {
 _____ nombre;
 _____ vida;
 _____ ataque;
 _____ recompensaOro;
 _____ esJefe; // verdadero o falso
}
```

Archivo `Mazmorra.java`:

```
public class Mazmorra {
 public static void main(String[] args) {

 Enemigo goblin = _____;
 goblin.nombre = "Goblin";
 goblin.vida = ____;
 goblin.ataque = ____;
 goblin.recompensaOro = ____;
 goblin.esJefe = _____;

 Enemigo dragon = _____;
 dragon.nombre = _____;
 dragon.vida = 300;
 dragon.ataque = 40;
 dragon.recompensaOro = 500.0;
 dragon.esJefe = _____;

 System.out.println("Enemigos en la mazmorra:");
 System.out.println("- " + _____ + " (vida " + _____ + ")");
 System.out.println("- " + _____ + " (vida " + _____ + ")");

 if (dragon._____) {
 System.out.println(dragon.nombre + " es un JEFE. ¡Cuidado!");
 }
 }
}
```

**Preguntas de control:**

1. ¿Cuántos objetos `Enemigo` existen en memoria al terminar el programa? \_\_\_\_\_
2. Si escribo `goblin.vida = 0;`, ¿cambia algo en `dragon`? ¿Por qué? \_\_\_\_\_
3. ¿Qué valor tendría `goblin.vida` si nunca lo asignara? \_\_\_\_\_

 **AHORA TÚ — La ficha completa del personaje (20 + 15 + 15 XP)**

**Misión 6.1:** Crea la clase `Personaje` con estos atributos: `nombre`, `clase`, `nivel`, `vida`, `vidaMaxima`, `ataque`, `defensa`, `oro`, `estaVivo`.

Luego, en un archivo `Aventura.java`:

1. Crea **tres** personajes: dos con datos fijos escritos por ti y uno pedido al jugador con `Scanner`.
2. Muestra la ficha de los tres.
3. Calcula e informa **cuál tiene el poder más alto**, donde `poder = ataque * 3 + defensa * 2 + nivel * 10`.
4. Simula un ataque: el personaje 1 golpea al personaje 3. El daño real es `ataque - defensa` (si da negativo, el daño es 0). Actualiza la vida y el estado `estaVivo`.
5. Muestra el estado de los tres **después** del ataque y demuestra que los otros dos no se vieron afectados.
6. Cura al personaje 3 en 30 puntos **sin pasarse de su `vidaMaxima`**.

**Pistas guiadas:**

-  Pista 1: cada personaje necesita **su propio `new`**. Tres personajes = tres `new`. Si escribes `Personaje p2 = p1;` no creaste nada, solo pusiste un segundo nombre al mismo objeto.
-  Pista 2: para comparar tres poderes, calcula los tres en variables `int` y usa la escalera `if / else if / else`. También puedes usar el patrón del máximo del Nivel 5.
-  Pista 3: el tope de curación es un `if` de una línea: `if (p3.vida > p3.vidaMaxima) { p3.vida = p3.vidaMaxima; }`

 **BONUS CREATIVO (10 XP):** agrega el atributo `arma` (String) y `esVenenoso` (boolean). Si el atacante tiene veneno, el defensor pierde 5 de vida adicionales.

 **CAZA DE BUGS #6 (25 XP)**

```
public class BugObjetos {
 public static void main(String[] args) {

 Heroe h1 = new Heroe();
 h1.nombre = "Aria";
 h1.vida = 100;

 Heroe h2 = h1;
 h2.nombre = "Kael";
 h2.vida = 150;

 Heroe h3;
```

```

 h3.nombre = "Nyx";

 System.out.println("h1: " + h1.nombre + " - " + h1.vida);
 System.out.println("h2: " + h2.nombre + " - " + h2.vida);
 }
}

```

1. **Bug 1:** el programa ni siquiera compila. ¿Qué línea lo impide y qué dice exactamente el compilador? \_\_\_\_\_  
¿Qué le falta a `h3`? \_\_\_\_\_
2. **Bug 2:** arregla el bug 1 y ejecuta. `h1` imprime `Kael - 150` en vez de `Aria - 100`. ¿Por qué? \_\_\_\_\_
3. Dibuja el diagrama de memoria de `h1` y `h2` como está el código: \_\_\_\_\_
4. Corrige el bug 2. ¿Cuántos `new` debe haber en total en un programa con tres héroes distintos? \_\_\_\_\_

## ESTACIÓN IA — Nivel 6 · EL FILTRO

Este es el reto de IA más importante del taller, porque aquí la IA **casi con seguridad** te va a dar algo que no puedes usar todavía.

1. Pídele: *"Crea una clase Heroe en Java y créame dos héroes."* (Esta vez **NO pegues el filtro**. A propósito.)
2. Mira su respuesta. Lo más probable es que traiga **constructor** (`public Heroe(String nombre, int vida)`), **getters y setters** (`getNombre()`, `setVida()`) y quizá **private**.
3. **Ese código es correcto, pero no es tuyo todavía.** Tu tarea: **reescribelo** usando solo lo que sabes — atributos sin **private**, `new Heroe()` vacío y asignaciones con punto.
4. Ahora sí, pídele lo mismo **con el filtro pegado**. ¿Obedeció? Si volvió a meter constructores, insiste y anota cuántos intentos hicieron falta.
5. Pregúntale: *"¿Qué es un constructor y por qué me facilitaría la vida aquí?"* — **léelo por curiosidad, no lo uses**. Es tu siguiente tema y ya vas a llegar preparado.

 Bitácora. Este ejercicio vale doble: **50 XP** si documentas bien la traducción de código avanzado a código que sí entiendes.

## CHECKPOINT NIVEL 6 — XP: \_\_\_\_\_ / 95

- ☐ Sé la diferencia entre clase (molde) y objeto (galleta)
- ☐ Entiendo qué hace exactamente `new`
- ☐ Sé que la variable guarda una **referencia**, no el objeto
- ☐ Sé por qué `Heroe h2 = h1`; no crea un segundo héroe
- ☐ Sé qué es un `NullPointerException` y por qué ocurre

## NIVEL 7 — EL EJÉRCITO

### Arrays de objetos

 45 minutos · Máx. 95 XP

### CARGA RÁPIDA

Ya tienes objetos. Ya tienes arrays. **Júntalos y tienes un ejército.**

```
Heroe[] equipo = new Heroe[3];
```

#### ATENCIÓN, ESTO ES LO MÁS IMPORTANTE DEL NIVEL:

Esa línea **NO crea tres héroes**. Crea un array con **tres casillas vacías**, y como los objetos valen **null** por defecto, el array queda así:

```
equipo → [null] [null] [null]
 0 1 2
```

Si escribes `equipo[0].nombre = "Aria"`; ahora mismo →  **NullPointerException**.

Hay que crear cada héroe, uno por uno:

```
Heroe[] equipo = new Heroe[3];

for (int i = 0; i < equipo.length; i++) {
 equipo[i] = new Heroe(); // ← ¡ESTA línea es la que crea el objeto!
 equipo[i].nombre = "Héroe " + (i + 1);
 equipo[i].vida = 100;
}
```

```
equipo → [•] [•] [•]
```



3 casillas + 3 objetos = 4 "new" en total  
(1 del array + 3 de los héroes)

### Acceder a un atributo dentro del array

```
equipo[1].vida = 50; // asignar
```

```
System.out.println(equipo[1].nombre); // Leer
equipo[0].oro += 100; // modificar
```

Léelo de izquierda a derecha: *"del equipo, el elemento 1, su vida"*.

## Recorrer un array de objetos

```
for (int i = 0; i < equipo.length; i++) {
 System.out.println(equipo[i].nombre + " - vida: " + equipo[i].vida);
}
```

🔑 **Compara esto con el Nivel 5.** Antes necesitabas `nombres[i]`, `vidas[i]` y `ataques[i]` coordinados a mano. Ahora `equipo[i]` es el héroe completo: nombre, vida, ataque, oro y todo lo demás viajan juntos. Esa es exactamente la razón por la que existen las clases.

## 👁️ TE LO MUESTRO — El escuadrón completo

```
import java.util.Scanner;

public class Escuadron {
 public static void main(String[] args) {

 Scanner sc = new Scanner(System.in);

 System.out.print("¿Cuántos héroes tendrá tu escuadrón? ");
 int cantidad = sc.nextInt();
 sc.nextLine();

 Heroe[] equipo = new Heroe[cantidad];

 // 1. CREAR y CARGAR cada héroe
 for (int i = 0; i < equipo.length; i++) {

 equipo[i] = new Heroe(); // sin esta línea: NullPointerException

 System.out.println();
 System.out.println("--- Héroe " + (i + 1) + " ---");

 System.out.print("Nombre: ");
 equipo[i].nombre = sc.nextLine();

 System.out.print("Vida: ");
 equipo[i].vida = sc.nextInt();

 System.out.print("Ataque: ");
 equipo[i].ataque = sc.nextInt();

 System.out.print("Oro: ");
 equipo[i].oro = sc.nextDouble();
 sc.nextLine();
 }
 }
}
```

```

 equipo[i].estaVivo = true;
 }

 // 2. MOSTRAR el escuadrón
 System.out.println();
 System.out.println("===== ESCUADRÓN =====");
 for (int i = 0; i < equipo.length; i++) {
 System.out.println("[" + i + "] " + equipo[i].nombre
 + " | vida " + equipo[i].vida
 + " | ataque " + equipo[i].ataque
 + " | oro " + equipo[i].oro);
 }

 // 3. ESTADÍSTICAS (patrones del Nivel 5, ahora sobre objetos)
 int sumaVida = 0;
 int posMasFuerte = 0;
 double oroTotal = 0;

 for (int i = 0; i < equipo.length; i++) {
 sumaVida += equipo[i].vida;
 oroTotal += equipo[i].oro;

 if (equipo[i].ataque > equipo[posMasFuerte].ataque) {
 posMasFuerte = i;
 }
 }

 double vidaPromedio = (double) sumaVida / equipo.length;

 System.out.println();
 System.out.println("Vida promedio : " + vidaPromedio);
 System.out.println("Oro del grupo : " + oroTotal);
 System.out.println("Más fuerte : " + equipo[posMasFuerte].nombre
 + " (ataque " + equipo[posMasFuerte].ataque + ")");

 // 4. UN ATAQUE ENEMIGO a todo el escuadrón
 System.out.print("Daño del ataque enemigo en área: ");
 int danioArea = sc.nextInt();

 int caidos = 0;

 for (int i = 0; i < equipo.length; i++) {
 equipo[i].vida -= danioArea;

 if (equipo[i].vida <= 0) {
 equipo[i].vida = 0;
 equipo[i].estaVivo = false;
 caidos++;
 System.out.println(equipo[i].nombre + " ha caído.");
 } else {
 System.out.println(equipo[i].nombre + " resiste con " + equipo[i].vida);
 }
 }

 System.out.println("Sobrevivientes: " + (equipo.length - caidos) + " de " + equipo.length);

 sc.close();
}
}

```

**Prueba de escritorio** (3 héroes: Aria 110/25, Kael 60/15, Nyx 90/35; daño en área = 70):

i	nombre	vida antes	vida después	estaVivo	caídos
0	Aria	110	40	true	0
1	Kael	60	0 (era -10)	false	1
2	Nyx	90	20	true	1

Más fuerte = **Nyx** (ataque 35) · Sobrevivientes = **2 de 3** ✅

## 👉 LO HACEMOS JUNTOS — La patrulla (10 XP)

```
public class Patrulla {
 public static void main(String[] args) {

 Heroe[] patrulla = _____ Heroe[_____]; // 4 casillas

 for (int i = 0; i < patrulla._____; i++) {
 patrulla[i] = _____; // ← La línea que casi todos olvidan
 patrulla[i].nombre = "Guardia " + (i + _____);
 patrulla[i].vida = 50 + (i * 10);
 patrulla[i].estaVivo = _____;
 }

 for (int i = 0; i < patrulla.length; i++) {
 System.out.println(patrulla[_____]._____ + " → vida " + patrulla[_____]._____);
 }
 }
}
```

### Preguntas:

1. ¿Cuántas veces aparece la palabra **new** en un programa correcto con 4 guardias? \_\_\_\_\_
2. ¿Qué error exacto ocurre si borras la línea `patrulla[i] = new Heroe();`? \_\_\_\_\_
3. ¿Qué vida tiene `patrulla[3]`? \_\_\_\_\_

## 👉 AHORA TÚ — El registro del gremio completo (20 + 15 + 15 XP)

**Misión 7.1:** Usando la clase **Personaje** del Nivel 6, escribe un programa que:

1. Pregunte cuántos miembros tiene el gremio y cree el array del tamaño correcto.
2. Pida todos los datos de cada miembro con un **for** (¡recuerda el **new** dentro del ciclo!).
3. Muestre la tabla completa del gremio.
4. Calcule y muestre:
  - El **oro total** del gremio y el **promedio** (con decimales correctos).
  - El miembro con **más vida** y el de **menos vida** (con sus nombres).
  - **Cuántos** miembros están por encima del nivel promedio.
5. **Busque** un miembro por nombre (usando `.equalsIgnoreCase()`) y muestre su ficha completa, o avise si no pertenece al gremio.

6. Aplique un **evento**: todos los miembros de nivel menor a 5 reciben 50 de oro de subsidio. Informe a cuántos les tocó.

#### Pistas guiadas:

-  Pista 1: la línea `equipo[i] = new Personaje();` va **como primera instrucción dentro del for**, antes de cualquier asignación. Es literalmente el error más común de este nivel.
-  Pista 2: para el máximo, guarda la **posición** (`posMax`), no el objeto. Así después accedes con `equipo[posMax].nombre`.
-  Pista 3: si el usuario pide 0 miembros, tu programa no debe explotar al calcular el promedio. Protege la división.

 **BONUS CREATIVO (10 XP)**: agrega un **combate en cadena**: el miembro 0 ataca al 1, el 1 al 2, y así hasta el último, que ataca al 0. Muestra el estado final de todos.



#### CAZA DE BUGS #7 (25 XP)

```
public class BugEjercito {
 public static void main(String[] args) {

 Heroe[] equipo = new Heroe[3];

 for (int i = 0; i < equipo.length; i++) {
 equipo[i].nombre = "Héroe " + i;
 equipo[i].vida = 100;
 }

 for (int i = 0; i <= equipo.length; i++) {
 System.out.println(equipo[i].nombre);
 }
 }
}
```

1. **Bug 1**: el programa se cae en la primera vuelta del primer ciclo. ¿Cuál es la excepción exacta y por qué ocurre? \_\_\_\_\_ ¿Qué línea falta y dónde va? \_\_\_\_\_
2. **Bug 2**: arregla el bug 1 y vuelve a ejecutar. Ahora se cae al final. ¿Qué excepción es y por qué? \_\_\_\_\_
3. ¿Cuál es el índice máximo válido de este array? \_\_\_\_\_
4. Corrige ambos y verifica que imprima los tres héroes.



Estos dos bugs juntos son **el 90 % de los errores de todo primer semestre con arrays de objetos**. Si los dominas, ya vas adelantado.



#### ESTACIÓN IA — Nivel 7 · EL SABOTEADOR

1. Escribe TÚ un programa con un array de objetos que funcione.
2. Pídele a la IA: *"Actúa como un usuario que quiere romper este programa. ¿Qué datos escribiría para que falle? No lo arregles, solo dime los ataques."*
3. Prueba cada ataque que te proponga. ¿Cuáles funcionaron?
4. Arregla tu programa para resistirlos y vuelve a pedirle que lo ataque.
5. Reflexión final para la bitácora: *¿qué tipo de fallos se le ocurrieron a la IA que a ti no? ¿Y cuáles se te ocurrieron a ti que a ella no?*

 Bitácora final.

## ✓ CHECKPOINT NIVEL 7 — XP: \_\_\_\_\_ / 95

- ☐ Sé que `new Heroe[3]` crea casillas vacías, no héroes
- ☐ Pongo `equipo[i] = new Heroe();` dentro del ciclo
- ☐ Sé leer `equipo[i].atributo` de izquierda a derecha
- ☐ Sé aplicar los patrones del Nivel 5 sobre arrays de objetos

## JEFE FINAL — "CÓDIGO DRAGÓN v1.0"

### Proyecto integrador

 90 minutos · 200 XP

### La situación

El estudio te llama:

*"Todo lo que hiciste funciona por separado. Ahora necesito UN juego. Y necesito que me lo puedas explicar a mí, que no sé programar."*

### Requisitos del sistema

Construye **un juego jugable** con dos archivos:

`Personaje.java` — clase con atributos: `nombre`, `clase`, `nivel`, `vida`, `vidaMaxima`, `ataque`, `defensa`, `oro`, `estaVivo`.

`CodigoDragon.java` — todo el juego dentro de `main`, con un menú `do-while`:

#	Opción	Debe usar
1	<b>Crear el escuadrón</b> — pregunta cuántos héroes, los crea con <code>new</code> en un ciclo y pide sus datos	arrays de objetos + Scanner
2	<b>Ver el escuadrón</b> — tabla con todos los héroes y su estado	recorrido de array de objetos
3	<b>Combate</b> — elige un héroe por su índice y pelea contra un enemigo hasta que uno caiga	<code>while</code> + condicionales
4	<b>Tienda</b> — compra pociones con <code>switch</code> , verifica oro, tope de vida	<code>switch</code> + <code>if</code>
5	<b>Estadísticas</b> — oro total, vida promedio, el más fuerte, el más débil, cuántos vivos	los 4 patrones del Nivel 5
6	<b>Buscar héroe por nombre</b> — muestra su ficha o avisa que no existe	búsqueda con bandera <code>-1</code>
7	<b>Salir</b> — muestra el resumen final de la partida	—

## Restricciones (aquí está la nota del proyecto)

- ✓ **Solo lo visto en el taller.** Nada de constructores, métodos propios, `ArrayList` ni herencia. Todo dentro de `main`.
- ✓ Textos comparados con `.equals()` o `.equalsIgnoreCase()`, **nunca** con `==`.
- ✓ Todo índice que venga del usuario debe **validarse** antes de usarse.
- ✓ Ninguna división sin protección contra cero.
- ✓ La vida nunca queda negativa ni supera `vidaMaxima`.
- ✓ Si el usuario elige una opción del menú antes de crear el escuadrón, el programa **avisa** en vez de reventar.
- ✓ El `sc.nextLine()` de limpieza donde corresponda.

## Entregables

Entregable	XP
Los dos archivos <code>.java</code> compilando y ejecutando	70
Prueba de escritorio de una partida completa (crear → combatir → tienda → salir)	30
<b>Diagrama de memoria</b> dibujado a mano del array de objetos tras crear 3 héroes	20
Lista de <b>5 casos borde</b> que probaste y qué pasó en cada uno	20
<b>Video/audio de 3 minutos</b> explicando tu juego como si le hablaras a alguien que no programa	40
Bitácora de IA completa de los 7 niveles	20

## 🏁 La regla de oro del jefe final

**Tienes permitido usar IA para TODO el jefe final.**

Pero en la sustentación de 3 minutos vas a explicarlo tú, y quien evalúe puede señalar **cualquier línea** y preguntarte por qué está ahí.

Si no puedes explicar una línea, esa línea vale 0.

Esta regla no existe para castigarte. Existe porque en tu primer empleo va a pasar exactamente esto: la IA te dará código y tu valor profesional será **saber si sirve o no**. Aquí estás entrenando eso.

## ANEXOS

### ANEXO A — archivos de Java

```
// ===== ESQUELETO =====
import java.util.Scanner;

public class Main {
 public static void main(String[] args) {
 // el código va aquí
 }
}

// ===== VARIABLES =====
int vida = 100;
double oro = 250.5;
String nombre = "Aria";
boolean vivo = true;
char inicial = 'A';

// ===== ENTRADA =====
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
double d = sc.nextDouble();
String s = sc.nextLine(); // línea completa
String p = sc.next(); // una palabra
sc.nextLine(); // limpieza tras nextInt/nextDouble
sc.close();

// ===== SALIDA =====
System.out.println("con salto");
System.out.print("sin salto");

// ===== DECISIONES =====
if (cond) { ... } else if (cond2) { ... } else { ... }

switch (variable) {
 case 1: ...; break;
 case 2: ...; break;
 default: ...;
}

// ===== CICLOS =====
while (cond) { ... }
```

```

do { ... } while (cond);

for (int i = 0; i < n; i++) { ... }

// ===== ARRAYS =====
int[] a = new int[5];
int[] b = {10, 20, 30};
a.length // tamaño (sin paréntesis)
a[0] // primer elemento
a[a.length - 1] // último elemento

for (int i = 0; i < a.length; i++) { ... }

// ===== CLASES Y OBJETOS =====
public class Heroe {
 String nombre;
 int vida;
}

Heroe h = new Heroe();
h.nombre = "Aria";
h.vida = 100;

Heroe[] equipo = new Heroe[3];
for (int i = 0; i < equipo.length; i++) {
 equipo[i] = new Heroe(); // ¡imprescindible!
 equipo[i].vida = 100;
}

```

**Operadores:** + - \* / % . == != > < >= <= . && || ! . ++ -- += -= \*= /=

**Strings:** .equals() .equalsIgnoreCase() .length() .toUpperCase() .trim()

## ANEXO B — Diccionario del compilador (traduce el error real)

Cuando Java te grita, casi siempre dice una de estas diez cosas:

Mensaje real	Qué significa de verdad	Qué revisar
cannot find symbol	Usaste un nombre que Java no conoce	¿Escribiste mal? ¿Mayúscula distinta? ¿Declaraste la variable?
',' expected	Falta un punto y coma	Casi siempre en la línea <b>anterior</b> a la que señala
reached end of file while parsing	Falta una llave }	Cuenta las llaves; el editor te ayuda a emparejarlas
class X is public, should be declared in a file named X.java	El archivo y la clase no se llaman igual	Renombra el archivo
incompatible types: String cannot be converted to int	Le diste texto a una variable numérica	¿Usaste <code>nextLine()</code> donde iba <code>nextInt()</code> ?

Mensaje real	Qué significa de verdad	Qué revisar
<code>variable X might not have been initialized</code>	Declaraste pero no asignaste	Dale un valor inicial
<code>NullPointerException</code> (en ejecución)	Usaste un objeto que no existe	Falta un <code>`new`</code>
<code>ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5</code>	Índice fuera del array	Usa <code>i &lt; length</code> , no <code>i &lt;= length</code>
<code>InputMismatchException</code>	El usuario escribió texto donde pedías número	Valida la entrada
<code>unreachable statement</code>	Hay código después de un <code>break</code> o <code>return</code>	Ese código nunca se ejecutaría

 **Regla de lectura de errores:** empieza SIEMPRE por el **primer** error de la lista y por el **número de línea**. Los demás suelen ser consecuencia del primero. Arregla uno, recompila, repite.

## ANEXO C — Los 14 errores más frecuentes

#	Error	Síntoma	Antídoto
1	Comparar Strings con <code>==</code>	El <code>if</code> nunca entra, sin mensaje de error	<code>.equals()</code> / <code>.equalsIgnoreCase()</code>
2	Trampa del <code>nextLine()</code>	El programa "se salta" una pregunta	<code>sc.nextLine()</code> de limpieza tras <code>nextInt()</code>
3	División entera	Promedios sin decimales	<code>(double)</code> antes de la división
4	<code>i &lt;= length</code> en arrays	<code>ArrayIndexOutOfBoundsException</code>	Siempre <code>i &lt; length</code>
5	Olvidar el <code>new</code>	<code>NullPointerException</code>	Declarar no es crear
6	<code>Heroe h2 = h1;</code>	Cambiar uno cambia los dos	Un objeto por cada <code>new</code>
7	Array de objetos sin instanciar	<code>NullPointerException</code> en la primera vuelta	<code>equipo[i] = new Heroe();</code> dentro del ciclo
8	<code>;</code> después del <code>if</code>	La condición deja de aplicar	Nunca <code>;</code> tras <code>if</code> , <code>while</code> , <code>for</code>
9	Olvidar el <code>break</code> en <code>switch</code>	Se imprimen varios casos	Un <code>break</code> por <code>case</code>
10	Falta la actualización del ciclo	El programa se congela	Revisa <code>i++</code>
11	Inicializar dentro del ciclo	El total siempre da lo mismo	Inicializaciones antes del ciclo
12	<code>maximo = 0</code> como valor inicial	El máximo sale 0 con negativos	Inicializa con el primer elemento
13	<code>length</code> vs <code>length()</code>	No compila	Arrays sin paréntesis, Strings con paréntesis

#	Error	Síntoma	Antídoto
14	Nombre de archivo ≠ nombre de clase	No compila	Renombra el archivo

## ANEXO D — Soluciones

 **Alto.** Si vienes aquí sin haber intentado, estás pagando un gimnasio para ver a otros levantar pesas. Regla: intenta 10 minutos, usa las pistas, intenta 5 más. Solo entonces mira.

### Nivel 0 — La bolsa de oro

```
String nombreObjeto = "Poción de vida";
double precioUnitario = 12.5;
int cantidad = 4;

double totalGastado = precioUnitario * cantidad;

System.out.println("Objeto: " + nombreObjeto);
System.out.println("Compraste " + cantidad + " unidades");
System.out.println("Gastaste: " + totalGastado + " monedas de oro");
```

1. Porque 12.5 tiene decimales y un `int` los descartaría. 2. No compila: en Java el separador decimal es el **punto**. 3. `"Total: " + 4 + 2` imprime `Total: 42` (concatena); `"Total: " + (4 + 2)` imprime `Total: 6`.

### Nivel 1 — La tienda del herrero

```
import java.util.Scanner;
new Scanner(System.in)
 .nextDouble()
 .nextDouble()
 .nextInt()
 .sc.nextLine();
 .nextLine()
 .precioEspada * cantidad
 .oroDisponible - costoTotal
 .sc.close();
```

La limpieza va **después del último número** (`cantidad`) y **antes del** `nextLine()` **del nombre**, porque ese `nextInt()` dejó el Enter pendiente en el buffer.

### Nivel 1 — Caza de bugs #1

`arma` queda vacío (`""`). El `nextInt()` toma el número pero **deja el Enter en el buffer**; el `nextLine()` lo encuentra, lo interpreta como una línea vacía terminada y devuelve `""` sin esperar.

**Corrección:** agregar `sc.nextLine();` justo después de `int nivel = sc.nextInt();`

Punto 4: con `next()` sí funcionaría, porque `next()` ignora los espacios y saltos de línea sobrantes — pero solo leería **una palabra**, así que "Espada de Fuego" quedaría en "Espada".

## Nivel 2 — El portero de la mazmorra

```
if (gremio.equalsIgnoreCase("si")) {
 nivelMinimo = 8;
} else {
 nivelMinimo = 10;
}

if (nivel >= nivelMinimo && vida > 0) { ... }
if (nivel < nivelMinimo) { ... }
if (vida <= 0) { ... }
```

Se usa `equalsIgnoreCase` porque `==` compara referencias, no contenido, y porque el usuario puede escribir "Si", "SI" o "si".

## Nivel 2 — Caza de bugs #2

1. `clase == "guerrero"` compara **referencias**, no contenido. El texto leído por Scanner es un objeto distinto del literal, aunque las letras coincidan. → `clase.equalsIgnoreCase("guerrero")`
2. El `;` después de `if (nivel >= 10)` cierra el `if` inmediatamente. El bloque `{ }` que sigue ya no depende de nada y **siempre** se ejecuta. → quitar el `;`
3. Con `nivel = 1` se imprimen **tres** mensajes por el *fall-through*: sin `break`, Java sigue ejecutando los casos siguientes. → poner `break;` al final de cada `case`.

## Nivel 3 — El entrenamiento

```
experiencia = 0; sesiones = 0; while (experiencia < 100) { experiencia += 5; sesiones++; }
sesiones
```

1. 20 sesiones.
2. Ciclo infinito: `experiencia` se queda en 0 y la condición nunca se vuelve falsa.
3. Versión `for`:

```
for (int sesiones = 1; sesiones <= 20; sesiones++) { ... }
```

Es más clara cuando el número de vueltas se conoce de antemano; el `while` es mejor cuando depende de una condición que puede cambiar.

## Nivel 3 — Caza de bugs #3

Falta la **actualización**: `i++`; justo después de `suma += i`; dentro del `while`.

Punto 3: si se incrementa **antes** de acumular, se suma 2+3+...+11 = **65** en vez de 55. El orden dentro del cuerpo también importa.

Punto 4: `for (int i = 1; i <= 10; i++) { suma += i; }` — con `for`, la actualización es parte de la sintaxis, así que olvidarla es imposible.

## Nivel 4 — Los daños del combate

```
danios.length · danios[0] · danios[danios.length - 1] · suma = 0 · for (int i = 0; i < danios.length; i++) · suma += danios[i]; · danios[i] · suma
```

1. Porque los humanos cuentan desde 1 y los arrays desde 0. 2. `ArrayIndexOutOfBoundsException`: intentaría leer `danios[6]`, que no existe. 3. El índice del último elemento es `5` (`length - 1`).

## Nivel 4 — Caza de bugs #4

1. Imprime 4 líneas (índices 0 a 3) y se cae en el índice 4 con `ArrayIndexOutOfBoundsException: Index 4 out of bounds for length 4`.
2. Un array de 4 elementos tiene índices **0 a 3**. La condición correcta es `i < vidas.length`.
3. `length` en arrays es un **atributo**, no un método: va sin paréntesis. En `String` sí es un método: `texto.length()`.

## Nivel 5 — El daño máximo

```
golpes[0] · posMax = 0 · for (int i = 1; ...) · if (golpes[i] > maximo) · maximo = golpes[i]; · posMax = i;
```

1. Reporta el **primero** (índice 1, turno 2), porque la condición es `>` estricta y el segundo 45 no es mayor que el máximo actual. 2. Cambiar `>` por `>=`. 3. Porque el elemento 0 ya se tomó como valor inicial; volver a compararlo consigo mismo no aporta nada.

## Nivel 5 — Caza de bugs #5

1. `maximo = 0` es un valor que no está en el array; como todas las temperaturas son negativas, ninguna supera a 0 y el máximo se queda en 0. → `int maximo = temperaturas[0];` y arrancar el ciclo en `i = 1`.
2. `suma / temperaturas.length` es `-27 / 4` entre **dos enteros** → división entera → `-6`, que al guardarse en un `double` imprime `-6.0` en vez de `-6.75`. → `double promedio = (double) suma / temperaturas.length;`
3. Con `-4, -8, -12, -8` la suma es `-32` y la división da exactamente `-8`, así que el resultado coincide por casualidad. **Un bug que a veces da el resultado correcto es más peligroso que uno que siempre falla**, porque pasa las pruebas y llega a producción.

## Nivel 6 — La clase Enemigo

```
public class Enemigo {
 String nombre;
 int vida;
 int ataque;
 double recompensaOro;
 boolean esJefe;
}
```

```
Enemigo goblin = new Enemigo();
...
```

```

Enemigo dragon = new Enemigo();
...
if (dragon.esJefe) { ... }

```

1. Dos objetos. 2. No cambia nada en `dragon`: son objetos independientes, creados con dos `new` distintos. 3. `0`, el valor por defecto de `int`.

## Nivel 6 — Caza de bugs #6

1. `h3.nombre = "Nyx"`; no compila: `variable h3 might not have been initialized`. A `h3` le falta `= new Heroe()`;
2. `Heroe h2 = h1`; **no copia el objeto, copia la referencia**. `h1` y `h2` son dos nombres para el mismo objeto, así que modificar por `h2` modifica lo que ve `h1`.
3. Diagrama: `h1` — y `h2` — apuntando a **un solo** cuadro con `nombre = "Kael"`, `vida = 150`.
4. Corrección: `Heroe h2 = new Heroe()`; Con tres héroes distintos debe haber **tres** `new`.

## Nivel 7 — La patrulla

```

new Heroe[4] · patrulla.length · patrulla[i] = new Heroe(); · (i + 1) · estaVivo = true; ·
patrulla[i].nombre · patrulla[i].vida

```

1. **Cinco** veces: una para el array y una por cada uno de los 4 guardias. 2. `NullPointerException` en la primera vuelta, porque las casillas valen `null`. 3. `patrulla[3].vida = 50 + 3*10 = 80`.

## Nivel 7 — Caza de bugs #7

1. `NullPointerException`: `new Heroe[3]` solo crea tres casillas con valor `null`; no hay objetos. Falta `equipo[i] = new Heroe()`; como **primera línea dentro del primer for**.
2. `ArrayIndexOutOfBoundsException`: `Index 3 out of bounds for length 3`, porque el segundo ciclo usa `i <= equipo.length`.
3. El índice máximo válido es **2**.

## ANEXO E — Rúbrica de evaluación

Criterio	Excelente (5.0)	Bueno (4.0)	Aceptable (3.0)	Insuficiente (≤2.5)
<b>Compila y ejecuta</b>	Sin errores ni advertencias; corre a la primera	Compila con advertencias menores	Compila tras correcciones guiadas	No compila
<b>Lógica y estructuras</b>	Elige la estructura óptima en cada caso y no hay redundancias	Resuelve bien con alguna redundancia	Resuelve el caso general, falla en bordes	La lógica no resuelve el problema
<b>Manejo de datos</b>	Valida toda entrada, protege índices y divisiones, controla topes	Valida lo esencial	Validación parcial	Cualquier dato raro lo rompe

## ANEXO F — Hoja de Ruta (tu tablero de XP)

Nivel	Juntos (10)	Ahora tú (20)	Funciona (15)	Prueba (15)	Bug (25)	Bonus (10)	Total
1 Scanner							
2 Decisiones							
3 Ciclos							
4 Arrays							
5 Estadísticas							
6 Objetos							
7 Arrays de objetos							
👑 Jefe final	—	—	—	—	—	—	/200
						GRAN TOTAL	/865

Página 67 de 68

ANEXO G — Bitácora de IA

#	Nivel	Prompt que usé	¿Qué respondió?	¿Usó algo prohibido?	Error detectado	Qué corregí yo
1				<input type="checkbox"/> Sí <input type="checkbox"/> No		
2				<input type="checkbox"/> Sí <input type="checkbox"/> No		
3				<input type="checkbox"/> Sí <input type="checkbox"/> No		
4				<input type="checkbox"/> Sí <input type="checkbox"/> No		
5				<input type="checkbox"/> Sí <input type="checkbox"/> No		
6				<input type="checkbox"/> Sí <input type="checkbox"/> No		
7				<input type="checkbox"/> Sí <input type="checkbox"/> No		

**Reflexión final (5 líneas):** ¿En qué te ayudó realmente la IA? ¿Cuántas veces te ofreció código que todavía no puedes usar? ¿Qué sabes hacer hoy que no sabías al empezar?